

TECNOLOGÍAS DE LA INFORMACIÓN

Minería de Datos

3 créditos

Profesor:



Ricardo Ordoñez Avila, Mg.

Titulaciones	Semestre
MINERÍA DE DATOS	Sexto

Tutorías: El profesor asignado se publicará en el entorno virtual de aprendizaje online.utm.edu.ec), y sus horarios de conferencias se indicarán en la sección Cronograma de Actividades

PERÍODO ABRIL 2026 – AGOSTO 2026

Índice

Tabla de contenido

Tema 1: Introducción y proceso KDD	5
1.1 Definiciones	5
1.2 Etapas del proceso KDD	8
Tema 2: Selección, limpieza y transformación de datos	14
2.1 Selección: Tipos y fuentes de datos	15
2.2 Limpieza: Procesos de limpieza, enriquecimiento y EDA	34
2.3 Transformación: Reducción, aumento, discretización.	48



✓ Organización de la lectura para el estudiante por semana del compendio

Semanas	Compendio
Semana 1	Páginas 3 – 13
Semana 2	Páginas 14 – 33
Semana 3	Páginas 34 – 47
Semana 4	Páginas 48 – 62

Resultado de aprendizaje de la Unidad I

Comprender los fundamentos de la minería de datos y el descubrimiento de conocimiento en bases de datos partiendo de procesos de selección, limpieza y transformación de datos.

Unidad I

Fundamentos y conceptos básicos de la Minería de Datos.

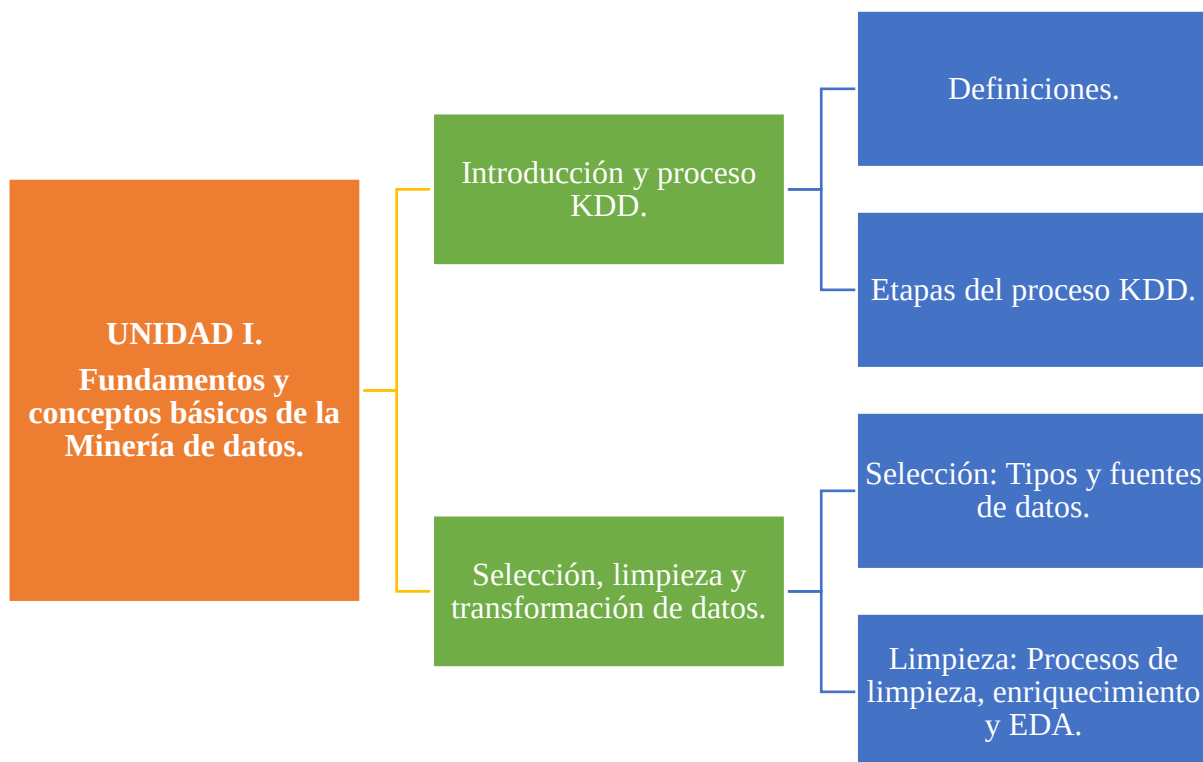
Aggarwal (2015), explica que la minería de datos es el estudio de la recopilación, la limpieza, el procesamiento, el análisis y la obtención de información útil a partir de los datos. Existe una amplia variedad en cuanto a los dominios de los problemas, las aplicaciones, las formulaciones y las representaciones de los datos que se encuentran en las aplicaciones reales. Por lo tanto, "minería de datos" es un término general que se utiliza para describir estos diferentes aspectos del procesamiento de datos.

Recientemente, todos los sistemas automatizados generan algún tipo de datos con fines de diagnóstico o análisis. Esto ha dado lugar a un diluvio de datos, que ha ido alcanzando el orden de los petabytes o exabytes. Algunos ejemplos de diferentes tipos de datos son los siguientes:

- **World Wide Web:** el número de documentos indexados en la web está ahora en el orden de miles de millones, lo que presenta una red muy extensa. Los accesos de los usuarios a estos documentos crean a su vez nuevos registros de acceso a la Web en los servidores y perfiles de comportamiento de los clientes, en los sitios comerciales. Además, los diferentes tipos de datos son útiles en diversas aplicaciones. Por ejemplo, los documentos de la Web y la información de enlaces pueden ser explotados para determinar las asociaciones entre diferentes temas en la Web. Por otro lado, los registros de acceso de los usuarios pueden ser minados para determinar patrones frecuentes de accesos o patrones inusuales de comportamiento, posiblemente injustificado.

- **Transacciones financieras:** son las más comunes cotidianamente, como el uso de un cajero automático (ATM), o una tarjeta de crédito. Ambos pueden crear datos de forma automatizada. De modo que, mediante la explotación de estos registros de transacciones, pueden ser útiles para el análisis de eventos como fraudes u otras actividades de diferente índole, sean estas comunes o poco comunes.
- **Interacciones de los usuarios:** Las interacciones de los usuarios generan grandes volúmenes de datos. Por ejemplo, el uso del dispositivo móvil crea con frecuencia un registro en la empresa proveedora de los servicios de telefonía, con detalles sobre la duración, destino y otros datos a partir de las llamadas. Muchas compañías de telecomunicaciones, analizan estos datos para determinar patrones de comportamiento de sus clientes, que permitan tomar decisiones y planificar campañas promocionales, analizar la capacidad de la red, los costos y demás aspectos de interés.

Ejes Temáticos



Tema 1: Introducción y proceso KDD

1.1 Definiciones

1.1.1 Datos históricos

Vallejo et al. (2018), explica que la minería de datos surge a principios de los años ochenta cuando la Administración de Hacienda de Estados Unidos desarrolló un programa de investigación para detectar fraudes en la declaración y evasión de impuestos, mediante lógica difusa, redes neuronales y técnicas de reconocimiento de patrones. Sin embargo, su expansión se produce hasta la década de los noventa, principalmente debido a:

- a) El incremento en la potencia de procesamiento de las computadoras, así como en la capacidad de almacenamiento.
- b) El crecimiento de la cantidad de datos almacenados se ve favorecido no solo por el abaratamiento de los discos y sistemas de almacenamiento masivo, sino también por la automatización de trabajos y técnicas de acopio de datos (observación con nuevas tecnologías, entrevistas más prácticas, encuestas por internet, etc.).
- c) La aparición de nuevos métodos y técnicas de aprendizaje y almacenamiento de datos, como las redes neuronales, la Inteligencia Artificial y el surgimiento del almacén de datos.

El propósito general de la minería de datos es analizar los datos desde todas las perspectivas estratégicas para la organización, con el fin de transformarla en información útil y conocimiento, siendo de utilidad general para aumentar la facturación, ampliar el margen operativo, etc. En general y, concretamente en las empresas, la minería de datos se soporta mediante utilidades informáticas, on-premises o en cloud, que sirven de instrumentos para el análisis de datos (Vallejo et al., 2018).



Recuerde que: El termino on-premises se refiere al hecho que los titulares de la licencia instalan el software en su propio entorno informático, sin recurrir a la nube o necesitar acceso a internet (IONOS, 2020).

1.1.2 La minería de datos, conceptos básicos.

La minería de datos es el proceso de detectar la información procesable de los conjuntos grandes de datos. Utiliza el análisis matemático para deducir los patrones y tendencias que existen en los datos. Normalmente, estos patrones no se pueden detectar mediante la exploración tradicional de los datos porque las relaciones son demasiado complejas o porque hay demasiados datos (Microsoft, 2022). Estos patrones y tendencias se pueden recopilar y definir como un modelo de minería de datos. Los modelos de minería de datos se pueden aplicar en escenarios como los siguientes:

- **Pronóstico:** cálculo de las ventas y predicción de las cargas del servidor o del tiempo de inactividad del servidor.
- **Riesgo y probabilidad:** elección de los mejores clientes para la distribución de correo directo, determinación del punto de equilibrio probable para los escenarios de riesgo, y asignación de probabilidades a diagnósticos y otros resultados.
- **Recomendaciones:** determinación de los productos que se pueden vender juntos y generación de recomendaciones.
- **Búsqueda de secuencias:** análisis de los artículos que los clientes han introducido en el carrito de la compra y predicción de posibles eventos.
- **Agrupación:** distribución de clientes o eventos en grupos de elementos relacionados, y análisis y predicción de afinidades.

La generación de un modelo de minería de datos forma parte de un proceso mayor que incluye desde la formulación de preguntas acerca de los datos y la creación de un modelo para responderlas, hasta la implementación del modelo en un entorno de trabajo (Microsoft, 2022).

1.1.3 Técnicas de la minería de datos

En el ámbito de la investigación las técnicas de minería de datos pueden ayudar a los científicos a clasificar y segmentar datos y a formar hipótesis. Según Clinic-Cloud (2016), las técnicas de data mining pueden ser de dos tipos:

Métodos descriptivos: Buscan patrones interpretables para describir datos, estas técnicas son: clustering, descubrimiento de reglas de asociación y descubrimiento de patrones secuenciales. Los métodos descriptivos se han utilizado, por ejemplo, para ver qué productos suelen adquirirse conjuntamente en el supermercado.

Asociación: Se trata de una de las técnicas más utilizadas. En esta técnica, una transacción y la relación entre los elementos se utilizan para identificar un patrón. Esta es la razón por la que también se conoce como «técnica de relación» (Bello, 2021). Se utiliza para realizar un análisis de la cesta de la compra, que se hace para conocer todos aquellos productos que los clientes compran juntos habitualmente, por ejemplo.

Agrupación o clustering: Esta técnica crea agrupaciones de objetos significativos que comparten las mismas características. A menudo se confunde con la clasificación, pero si comprendes correctamente cómo funcionan estas dos técnicas no tendrás ningún problema. A diferencia de la clasificación, que coloca los objetos en clases predefinidas, la agrupación en clústeres coloca los objetos en clases definidas por nosotros (Bello, 2021).

Patrones secuenciales: Esta técnica tiene como objetivo utilizar datos de transacciones y luego identificar tendencias, patrones y eventos similares en ellos durante un período de tiempo. Los datos históricos de ventas se pueden utilizar para descubrir artículos que los clientes compraron juntos en diferentes épocas del año (Bello, 2021). Las empresas pueden entender esta información recomendando a los clientes que compren esos productos en momentos en que los datos históricos no sugieren que lo harían. Las empresas pueden utilizar ofertas y descuentos para impulsar esta recomendación.

Métodos predictivos: Usan algunas variables para predecir valores futuros o desconocidos de otras variables. Son los siguientes: clasificación, regresión y detección de la desviación. Los métodos predictivos pueden emplearse en tareas como clasificar tumores en benignos o malignos.

Clasificación: Esta técnica tiene su origen en el machine learning. Clasifica elementos o variables en un conjunto de datos, en grupos o clases

predefinidos. Utiliza programación lineal, estadísticas, árboles de decisión y redes neuronales artificiales en la minería de datos, entre otras técnicas.

Predicción: Esta técnica predice la relación que existe entre las variables independientes y dependientes, así como las variables independientes por sí solas. Puede usarse para predecir ganancias futuras dependiendo de la venta. Supongamos que la ganancia y la venta son variables dependientes e independientes, respectivamente. Ahora, basándonos en lo que dicen los datos de ventas pasadas, podemos hacer una predicción de ganancias del futuro con una curva de regresión (Bello, 2021).

1.2 Etapas del proceso KDD

1.2.1 Antecedentes

El gran volumen de datos es el resultado directo de los avances tecnológicos y la informatización de todos los aspectos de la vida moderna. Por lo tanto, es natural examinar si se puede extraer información concisa y posiblemente procesable de los datos disponibles para objetivos específicos de la aplicación.

Aquí es donde entra la tarea de la minería de datos. Los datos brutos pueden ser arbitrarios, no estructurados, o incluso en un formato que no es inmediatamente adecuado para el procesamiento automatizado (Aggarwal, 2015). Por ejemplo, los datos recogidos manualmente pueden proceder de fuentes heterogéneas en formatos diferentes y, sin embargo, deben ser procesados por un programa informático automatizado para obtener información.

Para resolver este problema, los analistas de minería de datos utilizan una cadena de procesamiento, en la que los datos brutos se recogen, se limpian y se transforman en un formato estandarizado. Los datos pueden almacenarse en un sistema de base de datos comercial y procesarse finalmente para obtener información con el uso de métodos analíticos. De hecho, aunque la minería de datos suele evocar la noción de algoritmos analíticos, la realidad es que la mayor parte del trabajo está relacionada con la parte del proceso de preparación de los datos.

Esta cadena de procesamiento es conceptualmente similar a la de un proceso minero real desde un mineral hasta el producto final refinado. El término "minería" tiene su origen en esta analogía. Desde una perspectiva analítica, la minería de datos es un reto debido a la gran disparidad de problemas y tipos de datos que se encuentran. Por ejemplo, un problema de recomendación de productos comerciales es muy diferente de una aplicación de detección de intrusos, incluso a nivel del formato de los datos de entrada o de la definición del problema.

Incluso dentro de las clases de problemas relacionados, las diferencias son bastante significativas. Por ejemplo, un problema de recomendación de productos en una base de datos multidimensional es muy diferente de un problema de recomendación social debido a las diferencias en el tipo de datos subyacente.

Sin embargo, a pesar de estas diferencias, las aplicaciones de minería de datos suelen estar estrechamente relacionadas con uno de los cuatro problemas de la minería de datos: la minería de patrones de asociación; la agrupación; la clasificación; y, la detección de valores atípicos.

Estos problemas son tan importantes porque se utilizan como bloques de construcción en la mayoría de las aplicaciones de una forma indirecta u otra. Se trata de una abstracción útil porque nos ayuda a conceptualizar y estructurar el campo de la minería de datos de forma más eficaz. Los datos pueden tener diferentes formatos o tipos. El tipo puede ser:

- cuantitativo (por ejemplo, la edad),
- categórico (por ejemplo, el origen étnico),
- de texto,
- espacial,
- temporal u orientado a gráficos.

Aunque la forma más común de datos es la multidimensional, una proporción cada vez mayor pertenece a tipos de datos más complejos. Aunque existe una portabilidad conceptual de los algoritmos entre muchos tipos de datos a un nivel muy alto, no es así desde una perspectiva práctica. La realidad es que el tipo de datos preciso puede afectar el comportamiento de un algoritmo concreto de forma significativa. Como resultado, el analista de datos puede necesitar diseñar variaciones

refinadas del enfoque básico para datos multidimensionales, de modo que pueda ser utilizado efectivamente para un tipo de datos diferente.

De este modo, se establecen los antecedentes de la manipulación de grandes volúmenes de datos para obtener información objetiva para la toma de decisiones basadas en los datos. En ese sentido, el proceso de extraer conocimiento de grandes volúmenes de datos es reconocido por muchos investigadores, como un aspecto clave para la obtención potencial y útil de patrones novedosos a partir de los datos (Timaran et al., 2016).

1.2.2 El proceso KDD

El descubrimiento de conocimiento en bases de datos (KDD, del inglés Knowledge Discovery in Databases) es básicamente un proceso automático en el que se combinan descubrimiento y análisis (Timaran et al., 2016). El proceso consiste en extraer patrones en forma de reglas o funciones, a partir de los datos, para que el usuario los analice. Esta tarea implica generalmente preprocesar los datos, hacer minería de datos (data mining) y presentar resultados.

KDD se puede aplicar en diferentes dominios, por ejemplo, para determinar perfiles de clientes fraudulentos (evasión de impuestos), para descubrir relaciones implícitas existentes entre síntomas y enfermedades, entre características técnicas y diagnóstico del estado de equipos y máquinas, para determinar perfiles de estudiantes “académicamente exitosos” en términos de sus características socioeconómicas y para determinar patrones de compra de los clientes en sus canastas de mercado (Timaran et al., 2016).

1.2.3 Etapas KDD

El KDD, como proceso, involucra una secuencia de etapas claramente definidas, cada una fundamental para la transición de los datos en conocimiento. “Todos los sistemas de KDD mantienen la misma esencia, la minería de datos; su factor diferenciador radica en la implementación y la presentación” (Vieira et al., 2019, como se citó en Quiroz, 2012). Todos transitan las mismas etapas: recolección, depuración y análisis de datos, de donde se obtiene como resultado un “modelo descriptivo” que puede ser convertido en un “modelo predictivo”, de ser necesario.

En la figura 1, se presenta el proceso KDD propuesto por Timaran et al. (2016) con un ciclo iterativo, y a su vez interactivo. Este incluye varios pasos en los que interviene el usuario con la toma de muchas decisiones. Las etapas del proceso KDD, son:

- Selección;
- Preprocesamiento / limpieza;
- Transformación;
- Minería de datos (data mining); e,
- Interpretación / evaluación.

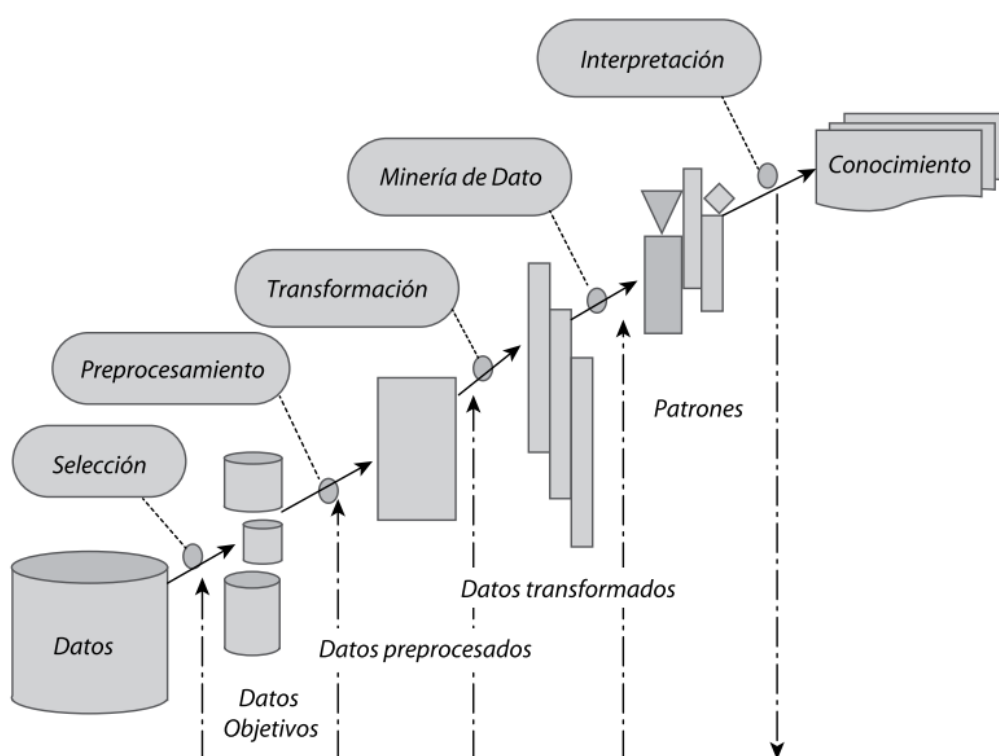


Figura 1. Etapas del proceso KDD, (Timaran et al., 2016).

Etapas de selección: En la etapa de selección, una vez identificado el conocimiento relevante y prioritario y definidas las metas del proceso KDD, desde el punto de vista del usuario final, se crea un conjunto de datos objetivo, seleccionando todo el conjunto de datos o una muestra representativa de este, sobre el cual se realiza el proceso de descubrimiento. La selección de los datos varía de acuerdo con los objetivos del negocio (Timaran et al., 2016).

Etapas de pre-procesamiento/limpieza: En la etapa de preprocesamiento/limpieza (data cleaning) se analiza la calidad de los datos, se aplican operaciones básicas como la remoción de datos ruidosos, se seleccionan estrategias para el manejo de datos desconocidos (missing y empty), datos nulos, datos duplicados y técnicas estadísticas para su reemplazo. En esta etapa, es de suma importancia la interacción con el usuario o analista.

Los datos ruidosos (noisy data) son valores que están significativamente fuera del rango de valores esperados; se deben principalmente a errores humanos, a cambios en el sistema, a información no disponible a tiempo y a fuentes heterogéneas de datos (Timaran et al., 2016). Los datos desconocidos empty son aquellos a los cuales no les corresponde un valor en el mundo real y los missing son aquellos que tienen un valor que no fue capturado. Los datos nulos son datos desconocidos que son permitidos por los sistemas gestores de bases de datos relacionales (SGBDR). En el proceso de limpieza todos estos valores se ignoran, se reemplazan por un valor por omisión, o por el valor más cercano, es decir, se usan métricas de tipo estadístico como media, moda, mínimo y máximo para reemplazarlos.

Etapas de transformación/reducción: En la etapa de transformación/reducción de datos, se buscan características útiles para representar los datos dependiendo de la meta del proceso. Se utilizan métodos de reducción de dimensiones o de transformación para disminuir el número efectivo de variables bajo consideración o para encontrar representaciones invariantes de los datos. Los métodos de reducción de dimensiones pueden simplificar una tabla de una base de datos horizontal o verticalmente.

La reducción horizontal implica la eliminación de tuplas idénticas como producto de la sustitución del valor de un atributo por otro de alto nivel, en una jerarquía definida de valores categóricos o por la discretización de valores continuos (por ejemplo, edad por un rango de edades).

La reducción vertical implica la eliminación de atributos que son insignificantes o redundantes con respecto al problema, como la eliminación de llaves, la eliminación de columnas que dependen funcionalmente (por ejemplo, edad y fecha de

nacimiento). Se utilizan técnicas de reducción como agregaciones, compresión de datos, histogramas, segmentación, discretización basada en entropía, muestreo, entre otras.

Etapas de minería de datos: El objetivo de la etapa minería de datos es la búsqueda y descubrimiento de patrones insospechados y de interés, aplicando tareas de descubrimiento como clasificación, patrones secuenciales, y asociaciones, entre otras. Las técnicas de minería de datos crean modelos que son predictivos o descriptivos.

Los modelos predictivos pretenden estimar valores futuros o desconocidos de variables de interés, que se denominan variables objetivo, dependientes o clases, usando otras variables denominadas independientes o predictivas, como por ejemplo predecir para nuevos clientes si son buenos o malos basados en su estado civil, edad, género y profesión, o determinar para nuevos estudiantes si desertan o no en función de su zona de procedencia, facultad, estrato, género, edad y promedio de notas.

Entre las tareas *predictivas* están la clasificación y la regresión. Los modelos descriptivos identifican patrones que explican o resumen los datos; sirven para explorar las propiedades de los datos examinados, no para predecir nuevos datos, como identificar grupos de personas con gustos similares o identificar patrones de compra de clientes en una determinada zona de la ciudad.

Entre las tareas *descriptivas* se cuentan las reglas de asociación, los patrones secuenciales, los clustering y las correlaciones. Por lo tanto, la escogencia de un algoritmo de minería de datos incluye la selección de los métodos por aplicar en la búsqueda de patrones en los datos, así como la decisión sobre los modelos y los parámetros más apropiados, dependiendo del tipo de datos (categóricos, numéricos) por utilizar.

Etapas de interpretación/evaluación de datos: En la etapa de interpretación/evaluación, se interpretan los patrones descubiertos y posiblemente se retorna a las anteriores etapas para posteriores iteraciones. Esta etapa puede incluir la visualización de los patrones extraídos, la remoción de los patrones redundantes o irrelevantes y la traducción de los patrones útiles en términos que sean entendibles para el usuario. Por otra parte, se consolida el conocimiento descubierto para

incorporarlo en otro sistema para posteriores acciones o, simplemente, para documentarlo y reportarlo a las partes interesadas; también para verificar y resolver conflictos potenciales con el conocimiento previamente descubierto (Timaran et al., 2016).



Sabías que: La minería de datos y KDD no hacen referencia al mismo concepto; el último circunscribe elementos más profundos, como la evaluación y la interpretación de los datos para, finalmente, encontrar el conocimiento (Quiroz, 2012).

Tema 2: Selección, limpieza y transformación de datos

El proceso de minería contiene muchas fases, como la limpieza de datos, la extracción de características y el diseño de algoritmos. El flujo de trabajo típico de minería de datos contiene las fases:

- Recopilación y selección de datos.
- Extracción de características y limpieza de datos
- Procesamiento analítico y algoritmos.

La recogida de datos puede requerir el uso de hardware especializado, como una red de sensores, trabajo manual, como la recogida de encuestas de usuarios. Aunque esta etapa es muy específica de la aplicación y a menudo está fuera del alcance del analista de minería de datos, es de vital importancia porque las buenas decisiones en esta etapa pueden afectar significativamente al proceso de minería de datos.

Después de la fase de recopilación, los datos suelen almacenarse en una base de datos o, más generalmente, en un almacén de datos para su procesamiento. Cuando se recogen los datos, a menudo no están en una forma adecuada para su procesamiento. En muchos casos, se obtienen diferentes tipos de datos. Para que los datos sean aptos para el procesamiento, es esencial transformarlos en un formato amigable para los algoritmos de minería de datos.

Es esencial extraer las características relevantes para el proceso de minería. La fase de extracción de características suele realizarse en paralelo a la limpieza de los datos, en la que se estiman o corrigen las partes que faltan o son erróneas. En muchos casos, los datos pueden extraerse de múltiples fuentes y deben integrarse en un formato unificado para su procesamiento. El resultado final de este procedimiento es un conjunto de datos bien estructurado, en un formato tabular. Tras la fase de extracción de características, los datos pueden volver a almacenarse en una base de datos para su procesamiento. La parte final del proceso de extracción es diseñar métodos analíticos eficaces a partir de los datos procesados (Aggarwal, 2015).

2.1 Selección: Tipos y fuentes de datos

En el proceso análisis en grandes volúmenes de datos, es vital importancia escoger las variables y características más adecuadas para presentar un buen algoritmo de minería de datos. Este problema se puede ver de diferentes enfoques, entre los más destacados: escoger los mejores atributos de los datos a partir de su análisis preliminar, eliminar los atributos redundantes o que aportan poca información al problema que se desea resolver, o reducir la dimensionalidad de los datos generando nuevos atributos a partir de atributos existentes. En cualquiera de estos casos, el objetivo es reducir el coste de espacio y computacional, que permita crear un modelo de similar o mejor calidad (Ayala, 2020).

La selección de atributos consiste en escoger únicamente aquellos atributos que son realmente, descartando otros que no aportan información al problema a resolver. Por otra parte, la extracción de atributos se trata de calcular nuevos atributos a partir de los existentes, de tal forma que los nuevos atributos resuman mejor la información que contienen, capturando la naturaleza de la estructura subyacente en los datos.

Existen métodos automáticos para la selección y extracción de atributos, no obstante, ambos métodos también pueden hacerse de forma manual. Es importante subrayar que la selección o extracción manual de atributos requiere de un experto que analice y escoja los atributos más relevantes. Este es un proceso ad hoc que

requiere gran conocimiento del dominio del problema, así como de los datos que se utilizarán para el proceso de minería de datos.

Por ejemplo, el índice de masa corporal, que se define como el peso de una persona en kilogramos dividido por el cuadrado de su altura en metros, informa mejor del grado de obesidad de una persona que las dos variables originales por separado. Este tipo de conocimiento proviene del contexto o dominio del problema, y no debe ser descartado. No obstante, en la mayoría de casos será necesario recurrir a los métodos automáticos para extraer características de un conjunto de datos (Ayala, 2020).

2.1.1 Variables y tipos de datos

En R prácticamente todos los datos pueden ser tratados como objetos, incluidos los tipos de datos más simples como son los números o los caracteres (Charte, 2014). Entre los tipos de datos disponibles están: vectores, matrices, factors, data frames y listas.

Los tipos de datos simples o fundamentales en R, son los siguientes:

- *numeric*: Todos los tipos numéricos, tanto enteros como en coma flotante y los expresados en notación exponencial, son de esta clase. También pertenecen a ellas las constantes Inf y NaN. La primera representa un valor infinito y la segunda un valor que no es numérico.
- *integer*: En R por defecto todos los tipos numéricos se tratan como double. El tipo integer se genera explícitamente mediante la función `as.integer()`.
- *character*: Los caracteres individuales y cadenas de caracteres tienen esta clase. Se delimitan mediante comillas simples o dobles.
- *logical*: Esta es la clase de los valores booleanos, representados en R por las constantes TRUE y FALSE.
- *factor*: un factor internamente se almacena como un número. Las etiquetas asociadas a cada valor se denomina niveles. Podemos crear un factor usando dos funciones distintas `factor()` y `ordered()`. El número de niveles de un factor se obtiene mediante la función `nlevels()`. El conjunto de niveles es devuelto por la función `level()`.

En la figura 2, se observa los tipos de variables categóricas o cualitativas y cuantitativas o numéricas. Las variables cualitativas pueden ser ordinales, aquellas que pueden ser ordenadas como el nivel de satisfacción (muy bueno, bueno, malo, muy malo) o variables nominales aquellas que representan generalmente categorías, por tanto, en las variables nominales el orden no es importante. Las variables de tipo cuantitativas, pueden ser discretas o continuas. Se dice que una variable es discreta cuando no puede tomar ningún valor entre dos consecutivos, y que es continua cuando puede tomar cualquier valor dentro de un intervalo (Quintela del Rio, 2019).

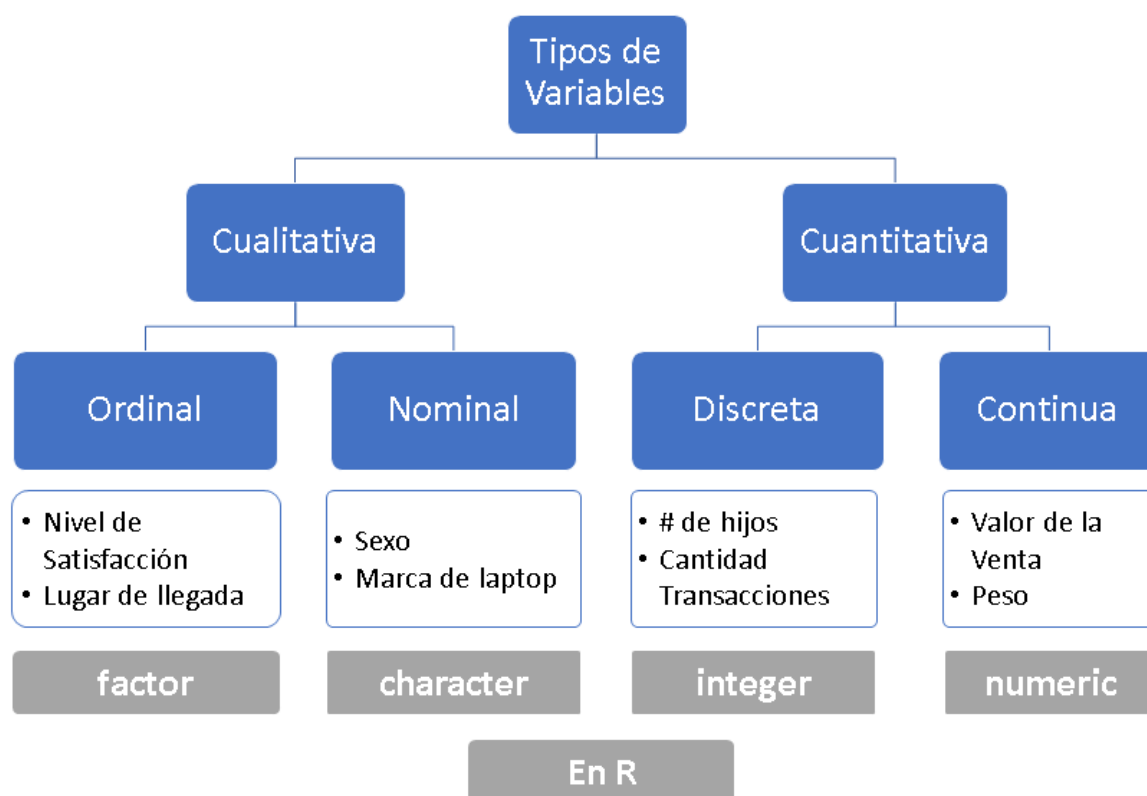


Figura 2. Tipos de variables.

Las variables en R no se definen ni declaran previamente a su uso, creándose en el mismo momento en que se les asigna un valor. El operador de asignación habitual en R es `<-`, pero también puede utilizarse `=` y `->` tal y como se aprecia en la figura 3. Introduciendo en cualquier expresión el nombre de una variable se obtendrá su contenido. Si la expresión se compone únicamente del nombre de la variable, ese contenido aparecerá por la consola. El símbolo `#` se utiliza para introducir un comentario. Todo lo que quede a la derecha de `#` no se ejecutará.

```

> a <- 45
> a

[1] 45

> a = 3.1416
> a

[1] 3.1416

> "Hola" -> a
> a

[1] "Hola"

variable1 <- 5000
variable1
variable2 <- "FCI"
variable2

## [1] 5000

## [1] "FCI"

# esto es un comentario
variable3 <- TRUE
variable3

## [1] TRUE
    
```

Figura 3. Operador de asignación y variables.



Sabías que: El operador de asignación habitual en R, puede ser generado con la combinación de teclas (alterna grande y el guion): **ALT –**

En la figura 4, se presentan ejemplos de variables de diferentes tipos:

```

variable1 <- 12.34 #numérico
variable2 <- "UTM" #carácter
variable3 <- FALSE #lógico
variable4 <- 4L #entero, agrega L al valor numérico
variable5 <- factor(c("hombre", "mujer", "mujer", "mujer", "hombre", "mujer"),
                    levels = c("hombre", "mujer"))
                    #factor sin orden
variable6 <- factor(c("alto", "bajo", "bajo", "medio", "alto"),
                    levels = c("bajo", "medio", "alto"),
                    ordered = TRUE)
                    #factor con orden
    
```

Figura 4. Ejemplos de variables y sus tipos de datos.

Datos: NA y NULL

En R, usamos NA para representar datos perdidos, mientras que NULL representa la ausencia de datos. La diferencia entre las dos es que un dato NULL aparece sólo cuando R intenta recuperar un dato y no encuentra nada, mientras que NA es usado para representar explícitamente datos perdidos, omitidos o que por alguna razón son faltantes. Por ejemplo, si tratamos de recuperar la edad de una

persona encuestada que no existe, obtendríamos un NULL, pues no hay ningún dato que corresponda con ello. En cambio, si tratamos de recuperar su estado civil, y la persona encuestada no contestó esta pregunta, obtendríamos un NA. NA además puede aparecer como resultado de una operación realizada, pero no tuvo éxito en su ejecución (Mendoza, 2018).

Verificar el tipo de un dato

Si no se conoce el tipo de dato, se puede usar la función **class()** para determinar el tipo de un dato, tal como se presenta en la figura 5. Esto es de utilidad para que las operaciones que deseamos realizar tengan los datos apropiados para ejecutarse con éxito. Esta función recibe como argumento un dato o vector y devuelve el nombre del tipo al que pertenece, en inglés.

```
class("3")
```

```
## [1] "character"
```

```
class(TRUE)
```

```
## [1] "logical"
```

Figura 5. El uso de la función class().

Otra de las formas de verificar el tipo de un dato, es mediante la verificación con la familia de funciones **is()**. En la figura 6, se presentan algunas funciones, y el tipo de dato que verifican. Estas funciones toman como argumento un dato, si este es del tipo que se verifica, devuelve como resultado **TRUE**, caso contrario devolverá **FALSE**.

Función	Tipo que verifican
<code>is.integer()</code>	Entero
<code>is.numeric()</code>	Numerico
<code>is.character()</code>	Cadena de texto
<code>is.factor()</code>	Factor
<code>is.logical()</code>	Lógico
<code>is.na()</code>	NA
<code>is.null()</code>	NULL

Figura 6. Verificación de datos con la función `is()`.

Si el objetivo es visualizar los objetos en R, se puede usar las funciones `cat()` o `print()`. La función `cat`, se puede utilizar para facilitar la impresión cuando hay muchas variables, y esta realiza mucha menos conversión que `print`. En la figura 7, se presenta un ejemplo.

```
cat(variable2)
print(variable6)

## UTM

## [1] alto bajo bajo medio alto
## Levels: bajo < medio < alto
```

Figura 7. Imprimir un dato.

Operadores en R

Los operadores en R, son símbolos que indican una operación a realizar, existen los tipos de operadores: aritmético, relacionales y lógicos; así como los operadores de asignación, ya estudiados en apartados anteriores. En la figura 8, se presentan los operadores aritméticos, estos se usan en operaciones como: suma, resta, multiplicación, división, potencia y división entera.

Show entries Search:

	Operador	Operación	Ejemplo	Resultado
1	+	Suma	5 + 3	8
2	-	Resta	5-3	2
3	*	Multiplicación	5 * 3	18
4	/	División	5/3	166667
5	^	Potencia	5 ^ 3	125
6	%%	División entera	5 %% 3	2

Showing 1 to 6 of 6 entries Previous Next

Figura 8. Operadores aritméticos.

Entre los operadores aritméticos, el de división entera o módulo requiere una explicación adicional sobre su uso. La operación que realiza es una división de un número entre otro, pero en lugar de devolver el cociente, nos devuelve el residuo. Por ejemplo, si hacemos una división entera de 4 entre 2, el resultado será 0. Esta es una división exacta y no tiene residuo (Mendoza, 2018).

Los operadores relacionales, son utilizados para realizar comparaciones y devolverán siempre un resultado TRUE o FALSE, esto es, VERDADERO o FALSO, respectivamente. En la figura 9, se despliega la lista de operadores utilizados en R.

Show entries Search:

	Operador	Comparación	Ejemplo	Resultado
1	<	Menor que	5 < 3	false
2	<=	Menor o igual que	5 <= 3	false
3	>	Mayor que	5 > 3	true
4	>=	Mayor o igual que	5 >= 3	true
5	==	Exactamente igual que	5 == 3	false
6	!=	No es igual que	5 != 3	true

Showing 1 to 6 of 6 entries Previous Next

Figura 9. Operadores relacionales.

Los operadores lógicos se utilizan con frecuencia, estos sirven para realizar operaciones de álgebra booleana, empleados para identificar las relaciones lógicas expresadas con valores TRUE o FALSE. En la figura 10 se amplía, los operadores a usar.

Show entries

Search:

	Operador	Comparación	Ejemplo	Resultado
1	<code>x y</code>	x Ó y es verdadero	<code>TRUE FALSE</code>	true
2	<code>x & y</code>	x Y y son verdaderos	<code>TRUE & FALSE</code>	false
3	<code>!x</code>	x no es verdadero (negación)	<code>!TRUE</code>	false
4	<code>isTRUE(x)</code>	x es verdadero (afirmación)	<code>isTRUE(TRUE)</code>	true

Showing 1 to 4 of 4 entries

Previous 1 Next

Figura 10. Operadores lógicos.

Estructuras de datos en R

Las estructuras de datos son objetos que contienen datos. Cuando trabajamos con R, lo que estamos haciendo es manipular estas estructuras. Las estructuras tienen diferentes características. Entre ellas, las que distinguen a una estructura de otra son su número de dimensiones y si son homogéneas o heterogéneas. En R todas las estructuras son tratadas como objetos. En la figura 11, se presentan las estructuras de uso frecuente: vector, matrices, data frame, listas.

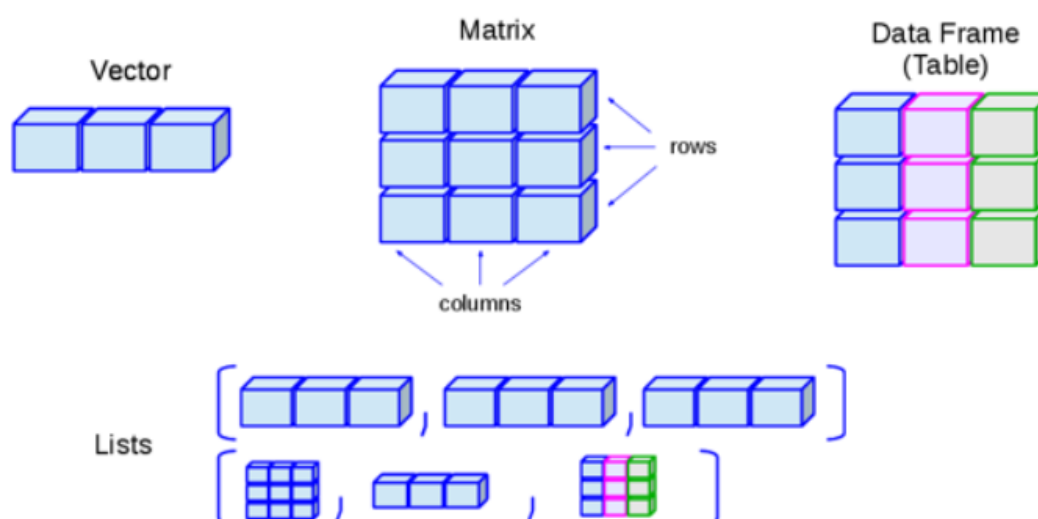


Figura 11. Estructura de datos en R.

VECTOR

Un vector es la estructura de datos más sencilla en R. Un vector es una colección de uno o más datos del mismo tipo. Todos los vectores tienen tres propiedades:

- **Tipo.** Un vector tiene el mismo tipo que los datos que contiene. Si tenemos un vector que contiene datos de tipo numérico, el vector será también de tipo numérico. Los vectores son atómicos, pues sólo pueden contener datos de un sólo tipo, no es posible mezclar datos de tipos diferentes dentro de ellos.
- **Largo.** Es el número de elementos que contiene un vector. El largo es la única dimensión que tiene esta estructura de datos.
- **Atributos.** Los vectores pueden tener metadatos de muchos tipos, los cuales describen características de los datos que contienen. Todos ellos son incluidos en esta propiedad. En este libro no se usarán vectores con metadatos, por ser una propiedad con usos van más allá del alcance de este libro (Mendoza, 2018).

Al referirnos a una estructura de datos básica en R que contiene un elemento de tipo similar, se hace referencia a que el vector es una estructura con datos homogéneos. El vector se puede crear utilizando la función **c()**. En R utilizando la función, **typeof()** se puede comprobar el tipo de datos del vector. La función **length()** nos da el número de elementos del vector. En la figura 12, se presenta un ejemplo de creación, impresión, tipo y longitud de un vector.

```
> #Introduction to Vectors in R
> #assigning values to vector
>
> vector1<-c(1,2,3,4,5)
>
> vector1 #checking values stored in vector 1
[1] 1 2 3 4 5
>
> typeof(vector1) #checking type of data stored in vector1
[1] "double"
>
> length(vector1) #checking length of vector1
[1] 5
> |
```

Figura 12. Aplicaciones con vectores.

En secuencia, un vector solo puede contener objetos de la misma clase (carácter, entero, etc.). La indexación se utiliza para extraer, actualizar los valores individuales o múltiples del vector. Para acceder a elementos del vector, usamos `[ ]`, `[1:5]`. Si se intenta acceder a posiciones no válidas, R devuelve NA (not available). La figura 13, muestra ejemplos de indexación.

```
> vector1[2] #selecting the 2nd element from vector
[1] 2
>
> vector1[2:4] #selecting 2nd to 4th elements of vector
[1] 2 3 4
>
> vector1[-3] #excluding the 3rd element from selection
[1] 1 2 4 5
```

Figura 13. Indexación en vectores.

La mayoría de las operaciones aritméticas son de carácter vectorial. Esto significa que R realiza las operaciones solicitadas elemento por elemento. En la figura 14, se muestra una operación de suma entre dos vectores.

```
A <- c(1,2,4,6)
B <- c(2,4,6,8)
Z <- A + B
Z

## [1] 3 6 10 14
```

Figura 14. Operaciones con vectores.

Cuando los vectores no tienen la misma longitud, R da una advertencia. La longitud debe ser igual para que se realice con éxito cualquier operación. En la figura 15, se presente un ejemplo de esto.

```
A <- c(1,2,4,6)
B <- c(2,4,6)
Z <- A + B

## Warning in A + B: longitud de objeto mayor no es múltiplo de la longitud de uno
## menor

Z

## [1] 3 6 10 8
```

Figura 15. Longitud en operaciones con vectores.

MATRICES

Las matrices y arrays pueden ser descritas como vectores multidimensionales. Al igual que un vector, únicamente pueden contener datos de un sólo tipo, esto es datos homogéneos, pero además de largo, tienen más dimensiones. En un sentido estricto, las matrices son un caso especial de un array, que se distingue por tener específicamente dos dimensiones, un “largo” y un “alto”.

Las matrices son, por lo tanto, una estructura con forma rectangular, con renglones y columnas. En R, una matriz puede considerarse como una unión de vectores. La función **cbind()** une columnas (vectores) en una matriz. En la figura 16, se crea una matriz con pesos y altura de personas, a partir de dos vectores.

```
peso <- c(68, 70, 80, 76)
altura <- c(165, 169, 175, 170)
matriz <- cbind(peso, altura)
matriz
```

```
##      peso altura
## [1,]   68    165
## [2,]   70    169
## [3,]   80    175
## [4,]   76    170
```

Figura 16. Creación de una matriz, a partir de vectores.

Para crear una matrices en R se usa la función **matrix()**. La función **matrix()** acepta dos argumentos, **nrow** y **ncol**. Con ellos especificamos el número de renglones y columnas que tendrá nuestra matriz, tal como se muestra en la figura 17.

```
matriz <- matrix(nrow=2, ncol=3) #crea matriz vacía
matriz
```

```
##      [,1] [,2] [,3]
## [1,]  NA  NA  NA
## [2,]  NA  NA  NA
```

Figura 17. Creación de una matriz, con argumentos **nrow** y **ncol**.

Las matrices se construyen en forma de columnas, por lo que las entradas comienzan en la "esquina superior izquierda y van hacia abajo". En la figura 18, se presenta una matriz creada con 12 datos numéricos, ordenados como se ha mencionado en este apartado.

```
matriz <- matrix(data=1:12, nrow=4, ncol=3)
matriz
```

```
##      [,1] [,2] [,3]
## [1,]  1   5   9
## [2,]  2   6  10
## [3,]  3   7  11
## [4,]  4   8  12
```

Figura 18. Creación de una matriz.

Para acceder a elementos de la matriz (indexación), usar `[ , ]` e indicamos la fila y columna a la que se quiere acceder. Si se quiere acceder a todas las columnas, basta con dejar el espacio vacío. También se puede especificar un rango de las filas o columnas para acceder. En la figura 19, se presentan estos escenarios.

```
matriz[1, 2] #accedemos a fila 1 columna 2
```

```
## [1] 5
```

```
matriz[1, ] #accedemos a fila 1 y todas las columnas
```

```
## [1] 1 5 9
```

```
matriz[2, 1:3] #accedemos a fila 2 y las tres primeras columnas
```

```
## [1]  2  6 10
```

Figura 19. Casos de indexación en matrices.

DATA FRAME

Los data frames son estructuras de datos de dos dimensiones (rectangulares) que pueden contener datos de diferentes tipos, por lo tanto, son datos heterogéneos. Esta estructura de datos es la más usada para realizar análisis de datos y seguro te resultará familiar si has trabajado con otros paquetes estadísticos. Los data frames pueden interpretarse como una versión más flexible de una matriz. Mientras que en una matriz todas las celdas deben contener datos del mismo tipo, las filas de un data frame admiten datos de distintos tipos, pero sus columnas conservan la restricción de contener datos de un sólo tipo.

En términos generales, los renglones en un data frame representan casos, individuos u observaciones, mientras que las columnas representan atributos, rasgos o variables. Por ejemplo, en la figura 20 se muestra los primeros cinco renglones del objeto iris, el famoso conjunto de datos Iris de Ronald Fisher, que está incluido en todas las instalaciones de R.

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1          5.1          3.5          1.4          0.2 setosa
## 2          4.9          3.0          1.4          0.2 setosa
## 3          4.7          3.2          1.3          0.2 setosa
## 4          4.6          3.1          1.5          0.2 setosa
## 5          5.0          3.6          1.4          0.2 setosa
```

Figura 20. Data frame IRIS.

Para crear un data frame se utiliza la función **data.frame()**. Es recomendable asignar un nombre a cada vector que compone al data frame. En la figura 21, se crea el data frame de nombre datos, con los vectores genero, peso, altura, soltero, que se convertirán en los encabezados de las columnas que formarán al data frame.

```
datos <- data.frame(genero = c("M", "F", "F", "M"),
  peso = c(58.7, 69.8, 80.1, 76.2),
  altura = c(174, 178, 169, 180),
  soltero = c(TRUE, TRUE, FALSE, TRUE))
datos
```

```
## genero peso altura soltero
## 1      M 58.7   174    TRUE
## 2      F 69.8   178    TRUE
## 3      F 80.1   169   FALSE
## 4      M 76.2   180    TRUE
```

Figura 21. Creación de un data frame.



Recuerde que: Un data frame está compuesto por vectores, y sus datos son heterogéneos. Por tanto, es posible asignar un nombre a cada vector, que se convertirá en el nombre de la columna.

Para la indexación en un data frame, se utiliza de igual manera que en las matrices, [,]. Sin embargo, también es posible acceder a grupos de elementos por nombre de columnas. La figura 22, presenta un ejemplo de indexación.

```
datos[ , "soltero"] #Se muestran todas las filas pero únicamente la columna soltero

## [1] TRUE TRUE FALSE TRUE
```

Figura 22. Indexación de un data frame.

Para conocer el número de las filas y columnas se usan las funciones siguientes: **nrow()** y **ncol()**. Para conocer los nombres de las filas y las columnas se usan las funciones siguientes: **rownames()** y **colnames()**. Si se requiere cambiar el nombre de columnas, basta con asignar un nuevo vector con tal información. En la figura 23, se presentan estos ejemplos.

```
nrow(datos) #número de filas

## [1] 4

ncol(datos) #número de columnas

## [1] 4

rownames(datos) #etiquetas de filas

## [1] "1" "2" "3" "4"

colnames(datos) #etiquetas de columnas

## [1] "genero" "peso" "altura" "soltero"

colnames(datos) <- c("sexo", "peso.en.kg", "estatura.en.cm", "civil")
datos

##   sexo peso.en.kg estatura.en.cm civil
## 1    M     58.7         174    TRUE
## 2    F     69.8         178    TRUE
## 3    F     80.1         169   FALSE
## 4    M     76.2         180    TRUE
```

Figura 23. Aplicaciones con filas y columnas del data frame.

Para acceder a columnas del data frame se usa el símbolo \$. Por ejemplo, **datos\$civil**, donde **datos** es el nombre del data frame, y **civil** es una variable o encabezado de columna. Cuando la cantidad de registros en el data frame es extenso, es posible visualizar únicamente las primeras o últimas las del mismo, usando las funciones **head()** y **tail()** respectivamente.

```
head(datos, 2) #muestra las dos primeras filas.
```

```
##  sexo peso.en.kg estatura.en.cm civil
## 1    M      58.7           174  TRUE
## 2    F      69.8           178  TRUE
```

```
tail(datos, 2) #muestra las dos últimas filas.
```

```
##  sexo peso.en.kg estatura.en.cm civil
## 3    F      80.1           169 FALSE
## 4    M      76.2           180  TRUE
```

Figura 24. Visualizar los primeros y últimos datos.

LISTAS

Las listas, al igual que los vectores, son estructuras de datos unidimensionales, sólo tienen largo, pero a diferencia de los vectores cada uno de sus elementos puede ser de diferente tipo o incluso de diferente clase, por lo que son estructuras heterogéneas. Las listas pueden contener datos atómicos, vectores, matrices, arrays, data frames u otras listas. Las listas se pueden crear explícitamente usando la función **list()** que toma un número arbitrario de argumentos.

```
lista <- list(c(2020, 2021), "Curso", TRUE, c(4L, 5L, 10L))
lista
```

```
## [[1]]
## [1] 2020 2021
##
## [[2]]
## [1] "Curso"
##
## [[3]]
## [1] TRUE
##
## [[4]]
## [1] 4 5 10
```

Figura 25. Ejemplo de lista.



Sabías que: Una lista puede ser considerada como un vector recursivo, pues es un objeto que puede contener objetos de su misma clase (Mendoza, 2018). Ver ejemplo en figura 26.

```
mi_vector <- 1:10
mi_matriz <- matrix(1:4, nrow = 2)
mi_df      <- data.frame("num" = 1:3, "let" = c("a", "b", "c"))

mi_lista <- list("un_vector" = mi_vector, "una_matriz" = mi_matriz, "un_df" = mi_df)

mi_lista
```

Figura 26. La lista como vector recursivo.

También es posible asignar etiquetas a cada componente de la lista. Para conocer todas las etiquetas de los componentes de la lista se usa la función `attributes()`. En la figura 27, se agregan etiquetas de ejemplo.

```
lista <- list(nombre="George", especie="Chelonoidis abingdonii",
antiguedad="110 anos", nacimientos=c(31, 36, 35, 20, 40))
lista
```

```
## $nombre
## [1] "George"
##
## $especie
## [1] "Chelonoidis abingdonii"
##
## $antiguedad
## [1] "110 anos"
##
## $nacimientos
## [1] 31 36 35 20 40
```

Figura 27. Etiquetas en los campos de la lista.

Por último, no es posible vectorizar operaciones aritméticas usando una lista, se nos devuelve un error como resultado.

2.1.2 Fuentes de datos

Un aspecto importante en el uso de RStudio enfocado en el análisis de datos de cualquier índole es el manejo de la fuente o el origen de los datos. Esto puede referir tanto a bases que quien efectúa el análisis haya construido como a bases de datos de estudio realizados por otros. La primera acción es realizar la descarga u obtención de una base de datos o fuente de datos, para ello existen algunos repositorios con los que se puede trabajar en conjunto con R. En R, el paquete específico llamado **haven**, cuenta con funciones para abrir bases de datos desde diferentes formatos, para ello se puede instalar el paquete en R Studio Desktop, mediante **install.packages("haven")**. Es importante señalar que al usar R en la web, mediante R Studio Cloud (<https://rstudio.cloud/>), o mediante un cuaderno de notas de R en Google Colaboratory (<https://colab.to/r>), ya no es necesario instalar paquetes, puesto que estas herramientas disponibles en la nube, no requieren de esta acción.

A continuación, se detallan, las alternativas disponibles para importar datos al acceder a las fuentes seleccionadas para la minería de datos:

Importar datos desde Excel. Para importar datos creados en Excel (.xlsx) se utiliza el paquete **openxlsx**. Mediante la función **read.xlsx**, se deberá agregar la ruta de origen del libro de Excel, tal como se muestra en la figura 28.

```
install.packages("readxl") #Descarga e instalación del paquete
library(readxl) #Cargar paquete en sesión de trabajo de R

CEP_excel <- read_excel("CEP_sep-oct_2017.xlsx") #Leer libro excel
```

Figura 28. Leer un archivo de extensión xlsx.

Importar datos de archivo de SPSS. Para importar un archivo generado de la aplicación de estadística SPSS, el cual posee una extensión (sav), se debe utilizar la librería **haven** mediante **library(haven)** en R. La función utilizada para este caso, es **read_spss**, tal como se presenta el ejemplo en la figura 29.

```
library(haven) #Debemos asegurarnos de que el paquete a ejecutar,
#está cargado en la sesión de Rstudio.

CEP <- read_spss("CEP_sep-oct_2017.sav") # Nombre del archivo entre comillas.
```

Figura 29. Leer un archivo de extensión sav.

Importar datos de archivo CSV. Otra forma de importar a R archivos del tipo hoja de cálculo es trabajar con el formato CSV (comma separated values) o archivo de valores delimitados por comas. Se trata de un tipo de archivo más simple que un libro de Microsoft Excel, en la medida que puede contener sólo una, y no varias hojas de trabajo. En la figura 30, se presenta un ejemplo de importación de archivo CSV, sin embargo, en ciertos casos es indispensable señalar el separador de columnas, mediante el argumento **sep = ‘;’**, y en otros inclusive, el separador decimal mediante **dec = ‘,’**, asumiendo para esta demostración que la coma (,) sería el separador decimal, y el punto y coma (;) para separar columnas.

```
aldo Clases/Usos de R")
> CEP_csv <- read.csv("CEP_sep-oct_2017.csv")
Error in read.table(file = file, header = header, sep = sep, quote = quote, :
  more columns than column names
> CEP_csv <- read.csv("CEP_sep-oct_2017.csv", sep = ";")
> View(CEP_csv)
```

Figura 30. Leer un archivo de extensión csv.

Descargar datos de la web. Es posible descargar archivos de internet usando R con la función `download.file()`. De esta manera tendremos acceso a una vasta diversidad de fuentes de datos. La función **download.file()** nos pide como argumento **url**, la dirección de internet del archivo que queremos descargar y **destfile** el nombre que asignaremos al archivo. Ambos argumentos como cadenas de texto, es decir, entre comillas.

```
download.file(
  url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data",
  destfile = "iris.data"
)
```

Figura 31. Descarga de datos de fuente de la web.

En la figura 31, se presentó un ejemplo, para descargar una copia del set iris disponible en el UCI Machine Learning Repository usamos la siguiente dirección como argumento url: <https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data>. Y asignamos “iris.data” al argumento dest.

Fuente de datos GitHub. Otra alternativa es el uso de un archivo colgado en un repositorio de GitHub (<https://github.com/>). GitHub se usa para alojar proyectos utilizando el sistema de control de versiones Git. Se puede crear repositorios, y en estos alojar los archivos de diferentes formatos que se podrían usar en la minería de datos. Con frecuencia se usan archivos de tipo csv, para acceder a este, se abre el archivo, y usando el botón RAW se captura su enlace. Posteriormente se podrá usar dicho enlace para leer un archivo de tipo csv, de ser el caso. En la figura 32, se crea un objeto URL, para almacenar el enlace del archivo sin procesar (RAW), luego se leerá y almacenará en el objeto DATOS, lo contenido en el archivo CARBOHIDRATOS_R, de tipo csv alojado en un repositorio de GitHub. Nótese, que se define la letra (T), para indicar que existirán encabezados de columnas (sep), el separador de columnas y el separador decimal (dec). Posterior a ello, se realiza una lectura de los datos para conocer los datos, mediante las funciones **dim** (24 filas, y dos columnas) y **str** (detalles del formato de las variables).

Subida de datos:

```
[ ] url <- 'https://raw.githubusercontent.com/erordonez/ESTADISTICA/main/CARBOHIDRATOS_R.csv'
    datos<-read.csv(url,T,sep=';',stringsAsFactors = TRUE, dec=',')
```

Verificación de datos:

```
[ ] dim(datos)
    str(datos)

24 · 2
'data.frame': 24 obs. of 2 variables:
 $ energia      : num  5.2 4.5 6 6.1 6.7 5.8 6.5 8 6.1 7.5 ...
 $ carbohidratos: Factor w/ 4 levels "avena","cebada",...: 4 4 4 4 4 4 2 2 2 2 ...
```

Figura 32. Trabajar con datos alojados en GitHub.

Finalmente, existen sitios web, como <https://archive.ics.uci.edu/ml/datasets.php> y <https://www.kaggle.com/> en los que se podrá encontrar datos para la práctica.

2.2 Limpieza: Procesos de limpieza, enriquecimiento y EDA

La limpieza de datos es la parte dentro del proceso de la Ciencia de Datos que consume más tiempo. Con frecuencia, se hace referencia a que el 80% del tiempo se consume en limpieza de datos. Los principios de limpieza de datos proveen un estándar para limpiar la data para no reinventar la rueda cada vez que tratamos de limpiar base de datos. La idea de la base de datos limpia es hacer el análisis de datos más fácil, permitiendo enfocarnos en entender el problema y no la logística de la data (Salazar, 2022). Una base de datos limpia tiene las siguientes características:

- Cada columna es una variable: una variable contiene todos los valores que miden el atributo (altura, peso, temperatura, etc.).
- Cada fila es una observación.
- Cada tipo de unidad observacional forma una tabla.

Luego del proceso de importación, es importante entender los datos. Se debe entonces, dar respuestas las siguientes interrogantes:

- ¿Qué representa cada columna?
- ¿Qué tipo de dato debería tener cada columna?
- ¿Qué granularidad o atomicidad tienen los datos?
- Si es que se tiene varios conjuntos de datos ¿Cómo se relacionan los datos?
- ¿A qué periodo de tiempo corresponden los datos?

Para realizar la exploración de los datos se usan las funciones: **dim()**, para ver la dimensión de la base de datos, y **head()**, para ver las primeras filas de la base de datos, tal como se muestra en la figura 33. Incluso podemos seleccionar la cantidad de filas que queremos ver por ejemplo **head(base, n=20)**, inclusive **tail()**, para ver las últimas filas. A este grupo de funciones para explorar los datos, se suma **glimpse()**, esta permite explorar las variables, conociendo su tipo y algunos datos a observar de forma preliminar.

```
head(datos_licor, n = 8)
```

```
##          PRODUCTO          LOCAL COD_CLTE          CLIENTE
## 1 Cerveza Pilsener light Samborondón 10027 Marilyn Marlene Arratia Rivas
## 2      Cerveza Pilsener Samborondón 10018  Xavier Rafael Lagos Oliva
## 3 Cerveza Pilsener light      Ceibos 10014  Irma del Carmen Gallardo Toledo
## 4      Cerveza Pilsener      Ceibos 10007  Honorio David Muñoz Molina
## 5 Cerveza Pilsener light Samborondón 10016  José Aquiles Aguayo Chávez
## 6      Cerveza Pilsener Samborondón 10020 Santos Alejandro Altamirano Pinto
## 7 Cerveza Pilsener light      Ceibos 10011  Francisco Segundo Ascencio Vera
## 8      Club verde Samborondón 10016  José Aquiles Aguayo Chávez
##  FECHA_FACTURA VENTA_UND VENTA_USD VENTA_COSTO CALIF_SERVICIO
## 1          42745          1        0.9        0.78          Mala
## 2          42738          2        1.6        1.40          Mala
## 3          42739          1        0.9        0.78        Regular
## 4          42776          4        3.2        2.80      Muy Buena
## 5          42738          6        5.4        4.68      Muy Buena
## 6          42750          4        3.2        2.80        Buena
## 7          42771          4        3.6        3.12      Muy Buena
## 8          42771          4        4.0        3.40        Buena
```

Figura 33. Mostrar los datos, indicando número de registros.

Para reconocer qué representa cada columna, se puede listar los nombres de las variables, mediante la función **names()**. Para saber qué tipo de dato debería tener cada columna se puede usar la función **glimpse()**. Para realizar operaciones sobre el tipo de dato que debería tener cada columna, se puede usar la familia de funciones **as**, por ejemplo: **as.date**, **as.numeric** y otros. Otras acciones como el hecho de cambiar el nombre de una columna en un data frame, se realiza con la función **rename()**.

El paquer **dplyr**, funciona para manipular datos de forma fácil. Este paquete tiene, entre otras, cinco funciones para manipular datos: **select()** **filter()** **arrange()** **mutate()** **summarize()**.

select()

Select nos permite seleccionar columnas. La sintaxis sería: **select(dataframe, col1, col2)** donde **col1**, **col2**, se refiere a los nombres de las columnas que queramos seleccionar.

La figura 34, muestra el caso 1, donde se realiza una consulta de los datos del dataframe de nombre morosidad, y se agrega a la derecha tres de las columnas que lo componen. Puede indicarse el nombre de las columnas que desee verificar.

```
select(morosidad, id, deuda17, deuda16)
```

```
## # A tibble: 78,703 x 3
##       id deuda17 deuda16
##   <dbl>   <dbl>   <dbl>
## 1      0  265838  133761
## 2  15452   83414   42569
## 3  34382    7808    4000
## 4  47329  620534   313886
## 5  50075  506818      NA
## 6 149005   8240      NA
## 7 469611  526936  264303
## 8 611086  103397   52546
## 9 776002   63089      NA
## 10 1070043 1451873  799625
## # ... with 78,693 more rows
```

Figura 34. Explorar datos con la función select(). Caso 1.

También podemos seleccionar todas las columnas menos algunas, esto lo hacemos poniendo - antes del nombre de la columna que no queremos seleccionar. Por ejemplo, si queremos todas las columnas menos Estado, como se muestra en la figura 35.

```
select(morosidad, -Estado)
```

```
## # A tibble: 78,703 x 9
##       id nombre.x deuda17
##   <dbl>   <chr>   <dbl>
## 1      0 VAN ADELBERGEN ISOLDA 265838
## 2  15452 BLAISE BLAISE CHRISTOPHE 83414
## 3  34382 INDUSTRIAS ELECTRONICAS COSTARRICENSES SA 7808
## 4  47329 VANNUCCI VANNUCCI ALBERTO 620534
## 5  50075 MARIA SELENIA CAYETANA CARVAJAL VENEGAS 506818
## 6 149005 ROSARIO CONEJO FÑCRUZ MARY GUASCH C 8240
## 7 469611 SAM SAM BRIAN WALTER HUBERT 526936
## 8 611086 MULLER MULLER JEAN-CLAUDE 103397
## 9 776002 RODRIGUEZ RODRIGUEZ VICTOR MANUEL 63089
## 10 1070043 MARKS RUIZ HENRY JAMES 1451873
## # ... with 78,693 more rows, and 6 more variables: situacion.x <chr>,
## #   lugar.pago.x <chr>, nombre.y <chr>, deuda16 <dbl>, situacion.y <chr>,
## #   lugar.pago.y <chr>
```

Figura 35. Explorar datos con la función select(). Caso 2.

filter()

La función filter permite filtrar filas. La sintaxis es: **filter(base, condicion)**. Donde agregamos la condición lógica para filtrar datos. Para ello usamos operadores lógicos. En la figura 36, se presenta un caso, donde se desea mostrar solamente las deudas superiores a un millón; y las deudas mayores a un millón y de difícil cobro.

```
filter(morosidad1, deuda17>1000000)
```

```
## # A tibble: 30,438 x 7
##       id                nombre.x  deuda17      situacion.x
##   <dbl>              <chr>    <dbl>      <chr>
## 1  1070043      MARKS RUIZ HENRY JAMES  1451873    COBRO JUDICIAL
## 2  3000465  HERNANDEZ RODAS MANUEL ANTONIO  2156724    COBRO ADMINISTRATIVO
## 3  4968006  SUCES DE EDUARDO ROJAS QUESADA  17668269    COBRO ADMINISTRATIVO
## 4  9700486    VERONICA ALVAREZ ZAMORAN  4177836    COBRO ADMINISTRATIVO
## 5  10000000    ANIBAL HERRERA BAUDELIO  1332517    COBRO ADMINISTRATIVO
## 6  11003242          RINE BORBON LUIS  1771856    COBRO JUDICIAL
## 7  11703874  MOSSUTO FONTANA ROCCO GIUSEPPE  1866968    COBRO JUDICIAL
## 8  11851989    PETER ALEXANDER TEODORI  1284362    COBRO JUDICIAL
## 9  11852243          DAMIA DAMIA EMIL  4685940    COBRO ADMINISTRATIVO
## 10 11880274  SCHLICKER SCHINDLBECK FRANK  1293144    DIFICIL COBRO
## # ... with 30,428 more rows, and 3 more variables: lugar.pago.x <chr>,
## # Estado <chr>, deuda16 <dbl>
```

Figura 36.1. Explorar datos con la función filter().

```
filter(morosidad1, deuda17>1000000 & situacion.x=="DIFICIL COBRO")
```

```
## # A tibble: 9,482 x 7
##       id                nombre.x  deuda17      situacion.x
##   <dbl>              <chr>    <dbl>      <chr>
## 1  11880274  SCHLICKER SCHINDLBECK FRANK  1293144    DIFICIL COBRO
## 2  12050037          DIAZ PILOTO JUSTO  22229809    DIFICIL COBRO
## 3  12251917          BIRD BIRD GEOFFREY  1267763    DIFICIL COBRO
## 4  13307917    LEWKOWICZ LEWKOWICZ HENRY  2074144    DIFICIL COBRO
## 5  13454769  BUSTOS VALDERRAMA JULIO ALBERTO  3317977    DIFICIL COBRO
## 6  13480637          PEREZ GIMENEZ VICENTE  1830254    DIFICIL COBRO
## 7  13509132    SEIDLER SEIDLER KARL PETER  4838378    DIFICIL COBRO
## 8  13952420    WILKIN WILKIN JEREMY ROBERT  1188119    DIFICIL COBRO
## 9  14050168          JANDREZ LOPEZ OVIDIO  4053836    DIFICIL COBRO
## 10 14110905  CETROLA CETROLA JULIO DOMINGO  1072692    DIFICIL COBRO
## # ... with 9,472 more rows, and 3 more variables: lugar.pago.x <chr>,
## # Estado <chr>, deuda16 <dbl>
```

Figura 36.2. Explorar datos con la función filter().

mutate()

Mutate nos permite crear nuevas columnas de forma fácil. En la figura 37, se presentan tres ejemplos. Podemos crear una variable que diga cuánto cambió la deuda, que es la diferencia entre deuda17 y deuda16, como se presenta en el ejemplo 1. También es posible crear una nueva variable que categorice el cambio en la deuda en si aumentó o no. Esto se realiza con la función **if_else()** o **ifelse** tal como se muestra en el ejemplo 2. Con mutate podemos crear multiples variables a la vez, separando cada una por coma, como se presenta en el ejemplo 3.

Ejemplo 1

```
morosidad1 <- mutate(morosidad1, cambio.deuda=deuda17-deuda16)
```

Ejemplo 2

```
morosidad1 <- mutate(morosidad1, tipo.cambio=ifelse(cambio.deuda<0,"disminuyó", "aumentó"))
```

Ejemplo 3

```
morosidad1 <- mutate(morosidad1, cambio.deuda=deuda17-deuda16,  
                      tipo.cambio=ifelse(cambio.deuda<0,"disminuyó", "aumentó"))
```

Figura 37. Crear nuevas columnas con la función mutate().

arrange()

Arrange permite ordenar las bases de datos por una o varias columnas. Por ejemplo, queremos ordenar la base en orden ascendente por deuda17 y por cambio.deuda. Si lo queremos en orden descendente usamos desc(). La figura 38, muestra ambos casos como ejemplo.

```
morosidad1 <- arrange(morosidad1, deuda17, cambio.deuda)
```

```
morosidad1 <- arrange(morosidad1, desc(deuda17), desc(cambio.deuda))
```

Figura 38. Ordenar los datos.

Simplificar el trabajo: %>%

Un operador muy útil cuando trabajamos con dplyr es **pipe operator** que visualmente se ve así %>%. Este operador nos va a facilitar muchísimo el trabajo con funciones y nos permite hacer comando con menos líneas de código.

¿Cómo funciona el %>%? Lo primero es poner el objeto (tabla o dataframe) al cual queremos aplicar las operaciones de la forma base %>% funcion(). Esto nos ahorra estar poniendo como primer argumento de las funciones de dplyr al objeto. Por ejemplo, recapitulemos todas las líneas de código que usamos anteriormente para limpiar la base de datos, en la figura 39.

```
morosidad1 <- select(morosidad, -nombre.y, -situacion.y, -lugar.pago.y)
morosidad1 <- filter(morosidad1, !is.na(deuda17), !is.na(deuda16))
morosidad1 <- mutate(morosidad1, cambio.deuda=deuda17-deuda16)
morosidad1 <- mutate(morosidad1, tipo.cambio=ifelse(cambio.deuda<0,"disminuyó", "aumentó"))
morosidad1 <- arrange(morosidad1, desc(deuda17), desc(cambio.deuda))
```

Figura 39. Operaciones para limpiar la base de datos.

Todos los pasos realizados anteriormente podríamos haberlos hecho de forma más simple usando el %>%, tal como se muestra en la figura 40.

```
morosidad2 <- morosidad %>%
  select(-nombre.y, -situacion.y, -lugar.pago.y) %>%
  filter(!is.na(deuda17), !is.na(deuda16)) %>%
  mutate(cambio.deuda=deuda17-deuda16,
         tipo.cambio=ifelse(cambio.deuda<0,"disminuyó", "aumentó")) %>%
  arrange(desc(deuda17), desc(cambio.deuda))
```

Figura 40. Operaciones para limpiar la base de datos, usando operador pipe.

Enriquecimiento de los datos

El enriquecimiento de datos, añade nuevos atributos, mientras que la transformación de datos cambia la forma o estructura de atributos de los datos existentes para satisfacer los requerimientos de la minería de datos.

Los pasos de enriquecimiento de datos y transformación de datos permiten modelar los datos hacia un formato más útil para la minería de datos, y en algunos casos los datos considerados marginales se convierten en datos útiles por esta manipulación (Calderón, 2006).

El enriquecimiento de datos es el proceso de añadir nuevos atributos, tales como campos calculados o datos de fuentes externas, a los datos ya existentes. La mayor parte de las referencias en la minería de datos tienden a combinar este paso con transformación de datos. La transformación de datos supone la manipulación de datos, pero el enriquecimiento de datos supone añadir información a los datos existentes. Esto puede incluir la combinación de los datos internos con los datos externos, que se obtiene de los distintos departamentos, compañías.

El enriquecimiento de datos es un paso importante si está intentando minar datos. Se puede añadir información a tales datos, de las fuentes de industria de partes externas normalizadas para hacer que el proceso de la minería de datos sea más exitoso y confiable, o proporcionar atributos derivados adicionales para una comprensión mejor de relaciones indirectas.

Análisis Exploratorio de datos (EDA)

El análisis exploratorio de datos no es un proceso formal regido por un conjunto estricto de reglas. El EDA es, más que nada, un estado mental. Durante las fases iniciales del EDA el analista de datos investiga todas las ideas que tenga presente en relación a sus datos. Algunas de estas ideas prosperarán, mientras que otras serán como callejones sin salida.

El EDA es una parte importante de cualquier análisis, y siempre se debe examinar la calidad de los datos. La limpieza de datos es una aplicación del EDA: se realiza preguntas acerca de si los datos cumplen con las expectativas o no, del analista. Para limpiar los datos se debe desplegar todas las herramientas del EDA: visualización, transformación y modelado (Wickham & Garrett, 2017).

En esta sección se conoce cómo usar la visualización y la transformación para explorar los datos de manera sistemática, una tarea que las personas de Estadística

suelen llamar análisis exploratorio de datos, o EDA (por sus siglas en inglés exploratory data analysis). El EDA es un ciclo iterativo en el que:

- Genera preguntas acerca de los datos.
- Busca respuestas visualizando, transformando y modelando los datos.
- Usa lo que ha aprendido para refinar las preguntas y/o generar nuevas interrogantes.

El objetivo principal del analista durante el EDA, es desarrollar un entendimiento de los datos. La manera más fácil de lograrlo es usar preguntas como herramientas para guiar el proceso de investigación. Cuando se formula una pregunta, esta orienta la atención hacia una parte específica del conjunto de datos y ayuda a decidir qué gráficos, modelos o transformaciones son necesarios (Wickham & Garrett, 2017).

El EDA es un proceso fundamentalmente creativo. Y como la mayoría de los procesos creativos, la clave para formular preguntas de calidad es generar una gran cantidad de preguntas. Es difícil hacer preguntas reveladoras al inicio del análisis pues aún no se conoce qué percepciones o conocimientos están contenidos en el conjunto de datos. Por otro lado, cada nueva pregunta que se plantea revelará un nuevo aspecto de los datos y las probabilidades de hacer un descubrimiento serán mayores. Se puede profundizar en las partes más interesantes de los datos. No hay reglas específicas sobre qué preguntas se deben plantear para guiar la investigación. Sin embargo, hay dos tipos de preguntas que siempre serán útiles para hacer descubrimientos dentro los datos. Puede formular dichas preguntas de la siguiente manera:

- ¿Qué tipo de variación existe dentro de cada una de las variables?
- ¿Qué tipo de covariación ocurre entre las diferentes variables?

Existen otras terminologías que se deben conceptualizar.

- Una variable es una cantidad, cualidad o característica medurable, es decir, que se puede medir.
- Un valor es el estado de la variable en el momento en que fue medida. El valor de una variable puede cambiar de una medición a otra.

- Una observación es un conjunto de mediciones realizadas en condiciones similares (usualmente todas las mediciones de una observación son realizadas al mismo tiempo y sobre el mismo objeto). Una observación contiene muchos valores, cada uno asociado a una variable diferente.
- Los datos tabulares (tabular data en inglés) son un conjunto de valores, cada uno asociado a una variable y a una observación. Los datos tabulares están ordenados si cada valor está almacenado en su propia “celda”, cada variable cuenta con su propia columna, y cada observación corresponde a una fila.

Variación

La variación es la tendencia de los valores de una variable a cambiar de una medición a otra. La variación de cada variable sigue un patrón específico y este puede revelar información interesante. La mejor manera de entender dicho patrón es visualizando la distribución de los valores de la variable. Cómo visualizar la distribución de una variable dependerá de si la variable es categórica o continua. En R las variables categóricas usualmente son guardadas como vectores de factores o de caracteres. Para examinar la distribución de una variable categórica, usa un gráfico de barras.

```
ggplot(data = diamantes) +  
  geom_bar(mapping = aes(x = corte))
```

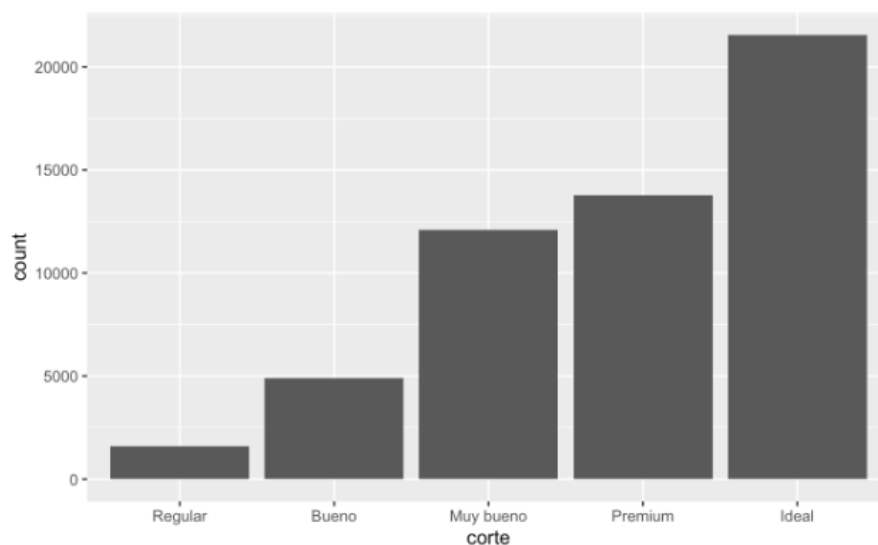


Figura 41. Gráfico de barras, para examinar la distribución.

La altura de las barras muestra cuántas observaciones corresponden a cada valor de x. Puedes calcular estos valores con `dplyr::count()`:

```

diamantes %>%
  count(corte)
#> # A tibble: 5 × 2
#>   corte      n
#>   <ord>   <int>
#> 1 Regular  1610
#> 2 Bueno   4906
#> 3 Muy bueno 12082
#> 4 Premium 13791
#> 5 Ideal   21551
  
```

Figura 42. Contador de valores en variables categóricas.

Una variable es continua si puede adoptar un set infinito de valores ordenados. Los números y fechas-horas son dos ejemplos de variables continuas. Para examinar la distribución de una variable continua, usa un histograma:

```

ggplot(data = diamantes) +
  geom_histogram(mapping = aes(x = quilate), binwidth = 0.5)
  
```

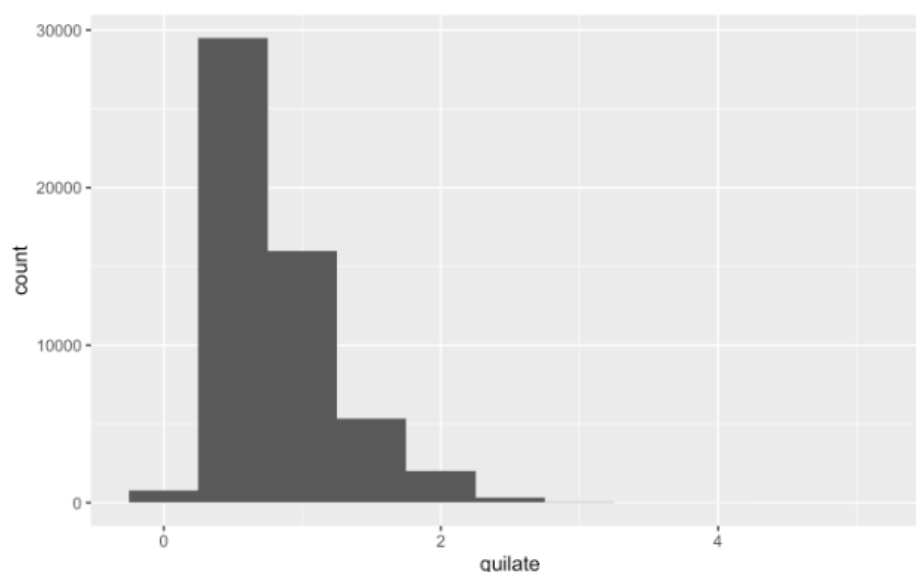


Figura 43. Histograma para analizar variables continuas.

Si se quiere sobreponer múltiples histogramas en la misma gráfica, te recomendamos usar `geom_freqpoly()` (*polígonos de frecuencia*) en lugar de `geom_histogram()`. `geom_freqpoly()` realiza el mismo cálculo que `geom_histogram()`, pero usa líneas en lugar de barras para mostrar los totales. Es mucho más fácil entender líneas que barras que se superponen.

```
ggplot(data = pequenos, mapping = aes(x = quilate, colour = corte)) +  
  geom_freqpoly(binwidth = 0.1)
```

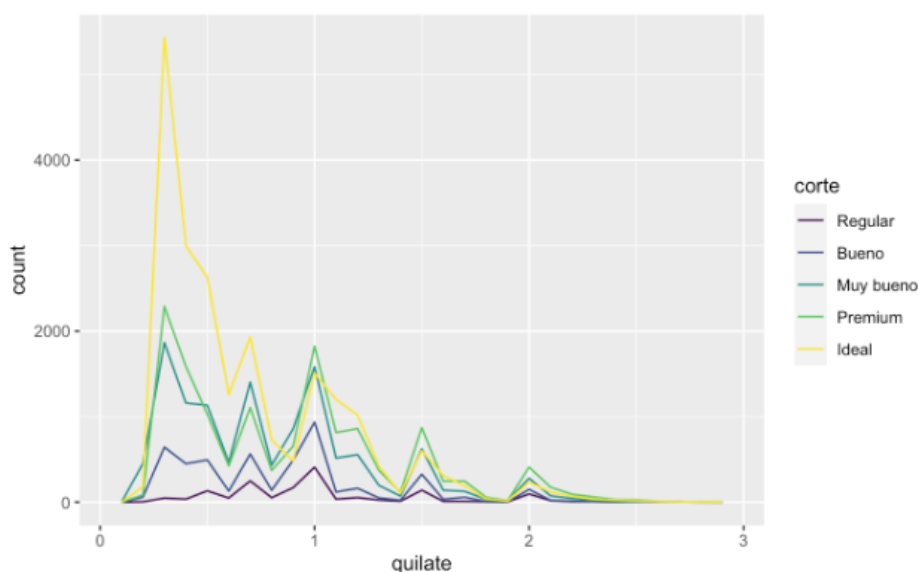


Figura 44. Polígono de frecuencia.

Valores inusuales: Los valores atípicos, conocidos en inglés como *outliers*, son puntos en los datos que parecen no ajustarse al patrón. Algunas veces dichos valores atípicos son errores cometidos durante la ingesta de datos; otras veces sugieren nueva información. Cuando tienes una gran cantidad de datos, es difícil identificar los valores atípicos en un histograma. Por ejemplo, toma la distribución de la variable y del set de datos de diamantes. La única evidencia de la existencia de valores atípicos son límites inusualmente anchos en el eje horizontal. Hay tantas observaciones en las barras comunes que las barras infrecuentes son tan cortas que no es posible verlas a simple vista (aunque quizá si observas con mucha atención el 0 podrías encontrar algo). Para facilitar la tarea de visualizar valores inusuales, necesitamos acercar la imagen a los valores más pequeños del eje vertical con `coord_cartesian()`.

```
ggplot(diamantes) +  
  geom_histogram(mapping = aes(x = y), binwidth = 0.5) +  
  coord_cartesian(ylim = c(0, 50))
```

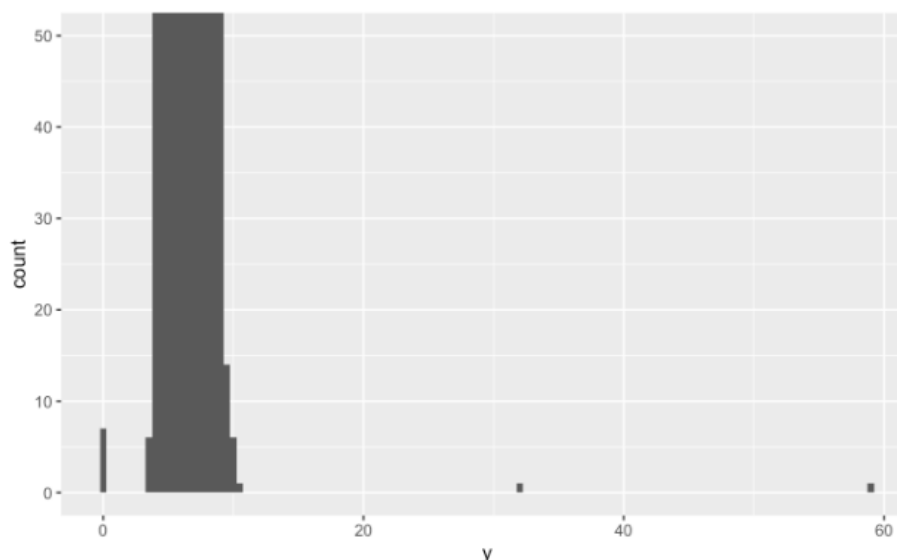


Figura 45. Valores inusuales (outliers).

Valores faltantes: Si se ha encontrado valores inusuales en el set de datos y simplemente quieres seguir con el resto de tu análisis, tienes dos opciones. Desecha la fila completa donde están los valores inusuales. La misma que no es siempre recomendada, porque el hecho de que una medida sea inválida no significa que todas las mediciones lo sean. Además, si la calidad de los datos es baja, después de aplicar esta estrategia en todas las variables ¡quizás se reducen los datos con los que se deba trabajar! En lugar de eso, se recomienda reemplazar los valores inusuales con valores faltantes. La manera más fácil de hacerlo es usar `mutate()` para reemplazar la variable con una copia editada de la misma. Se puede usar la función `ifelse()` para reemplazar valores inusuales con NA.

Covariación

Si la variación describe el comportamiento dentro de una variable, la covariación describe el comportamiento entre variables. La covariación es la tendencia de los valores de dos o más variables a variar simultáneamente de una manera relacionada.

La mejor manera de reconocer que existe covariación en los datos es visualizar la relación entre dos o más variables. Cómo hacerlo dependerá de los tipos de variables involucradas. En la figura 46, se presente la densidad, que es lo mismo que una cuenta estandarizada de manera que el área bajo cada polígono es igual a uno.

```
ggplot(data = diamantes, mapping = aes(x = precio, y = ..density..)) +  
  geom_freqpoly(mapping = aes(colour = corte), binwidth = 500)
```

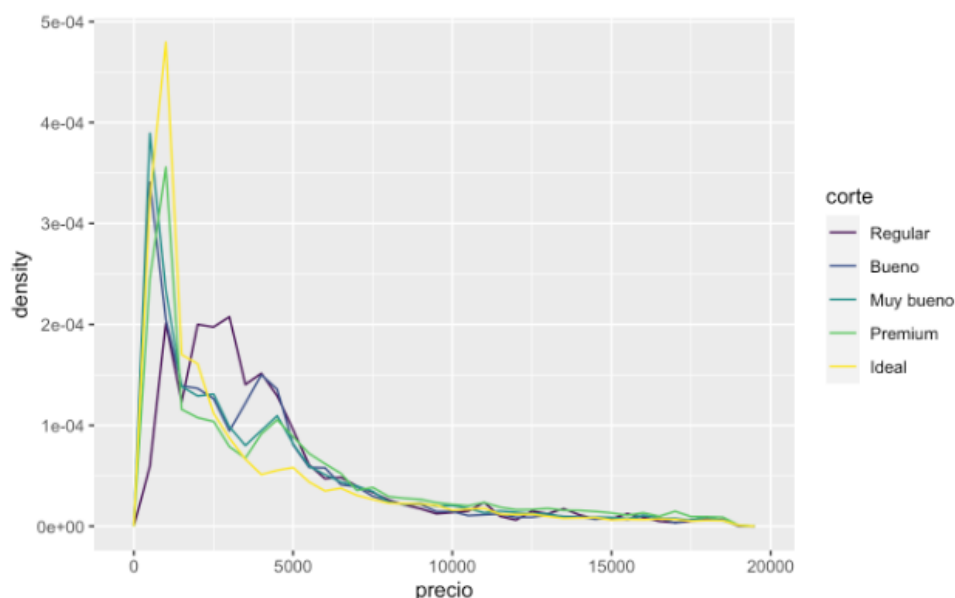


Figura 46. Densidad en polígonos de frecuencia.

Otra alternativa para mostrar la distribución de una variable continua agrupada en relación con otra variable categórica es el diagrama de caja (boxplot en inglés). Un diagrama de caja es un tipo de representación visual para la distribución de valores que es popular entre las personas que se dedican a la estadística. Cada diagrama de caja está integrado por una caja que comprende desde el percentil 25 de la distribución hasta el percentil 75, distancia que se conoce como rango intercuartil (interquartile range o IQR en inglés). En el centro de la caja se encuentra una línea que señala la mediana (es decir, el percentil 50), de la distribución. Estas tres líneas dan una idea sobre la dispersión de la distribución, así como también mostrarán si la distribución es simétrica en torno a la mediana o si está sesgada hacia uno de los lados.

Los puntos que representan observaciones que se encuentran a más de 1.5 veces el IQR a partir de cualquier borde de la caja. Estos puntos son inusuales en los datos, de manera que son graficados individualmente. Una línea (o bigote) que se extiende a partir de cada borde de la caja hasta el punto más lejano de la distribución que no sea considerado como un valor atípico.

```
ggplot(data = diamantes, mapping = aes(x = corte, y = precio)) +  
  geom_boxplot()
```

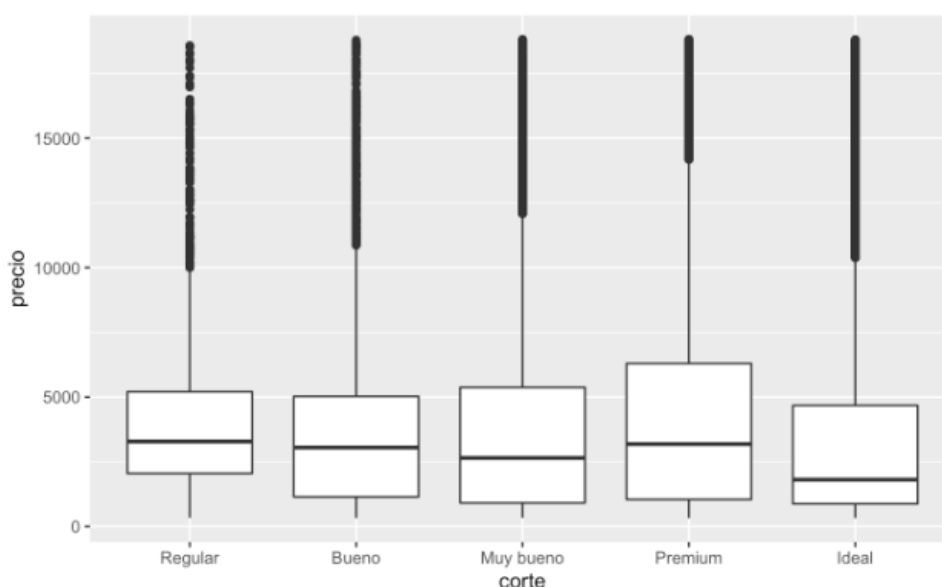


Figura 47. Diagrama de cajas y bigotes.

Si los nombres de las variables son muy largos, `geom_boxplot()` funcionará mejor si giramos el gráfico en 90°. Esto es posible agregando `coord_flip()` (*voltear coordenadas*).

```
ggplot(data = millas) +  
  geom_boxplot(mapping = aes(x = reorder(clase, autopista, FUN = median), y = autopista)) +  
  coord_flip()
```

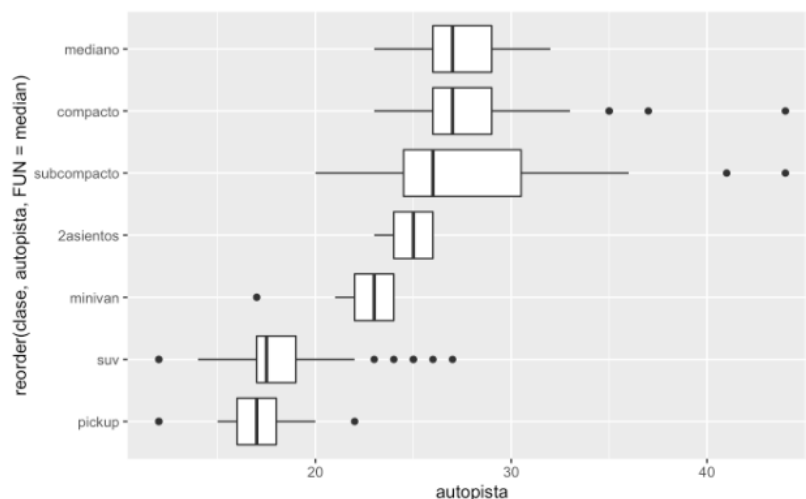


Figura 48. Voltear coordenadas en diagramas de cajas y bigotes.

2.3 Transformación: Reducción, aumento, discretización.

La transformación de datos, desde el punto de vista de minería de datos, es el proceso de cambiar la forma o estructura de los datos existentes. Una de las formas más comunes de la transformación de datos usadas en la minería de datos es la conversión de atributos continuos en atributos discretos.

Muchos algoritmos de minería de datos funcionan mejor al trabajar con un número pequeño de atributos discretos, tales como rangos de sueldos, antes que atributos continuos, tales como sueldos reales. Este paso, como con otros pasos de transformación de datos, no añade información para los datos, ni limpia los datos; en vez de ello hace que los datos sean fáciles de modelar. Ciertos proveedores de algoritmos de minería de datos pueden transformar los datos, en datos discretos de forma automática, usando una variedad de los algoritmos diseñados para crear rangos discretos basadas en la distribución de datos dentro de un atributo continuo. Demasiados valores discretos dentro de un atributo sencillo pueden sobrecargar ciertos algoritmos de minería de datos. Por ejemplo, usando códigos postales de las direcciones de cliente para categorizar clientes por la región son una técnica excelente si se propone examinar una región pequeña. Si, por el contrario, se planea

examinar los modelos de los clientes para un país entero, usando códigos postales pueden llevar a 50,000 o los valores más discretos dentro de un atributo sencillo; se debe usar un atributo con un alcance más ancho, tal como la ciudad o la información de estado suministrados por la dirección.

2.3.1 Expresiones regulares

Son patrones utilizados para encontrar una determinada combinación de caracteres dentro de una cadena de caracteres. Son como un lenguaje especial para hablar con nuestro computador. En esta sección se usa el paquete string.



Figura 48. Cadena de caracteres, composición.

Buscadores de coincidencias

- \d** cualquier dígito
- \w** cualquier caracter de palabra
- \s** espacio en blanco
- \b** límite de palabra
- .** cualquier caracter, excepto un salto línea
- \D** cualquier NO dígito
- \W** cualquier NO caracter de palabra
- \S** NO espacio en blanco

Ejemplo:

Encontrar un número: **\d**

Cuanticadores

Para indicar cuántas veces se quiere encontrar cada patrón.

? 0 o 1 vez

+ 1 o más

* 0 o más

Para indicar cuántas veces se quiere encontrar cada patrón.

{n} exactamente n

{n,} n o más

{n,m} entre n y m

{,m} hasta m

Ejemplo

Encontrar una secuencia de cuatro números: `\d{4}`

Anclas

Para fijar el inicio y/o término de una cadena.

^ inicio de la cadena

\$ n de la cadena

Ejemplo

Encontrar una secuencia de cuatro números que estén al final de una cadena de texto: `\d{4}$`

Alternos

Para indicar alternativas al encontrar patrones.

| Ó

[] uno de

Ejemplo

Encontrar la palabra minería con o sin tilde en la i: `miner[i|í]a`

En R, a todas las expresiones regulares que usan \ se les debe agregar otra \ para que

sean interpretadas correctamente.

Expresión regular en R

`\\d`

`\\s`

`\\w`

Para interpretar caracteres especiales. Para que R lo reconozca como tal

`\\^`

`\\$`

`\\+`

`\\?`

`\\*`

Escenarios de transformación empleando expresiones regulares: En la figura 49, se presenta un ejemplo de filtrado, donde se busca todas las versiones del dato “Valparaíso”.

```

telefonos %>%
  filter(str_detect(ciudad, "[v|V]alpara[i|i]so"))
    
```

```

## # A tibble: 9 x 4
##   nombre                ciudad      numero_telefonico nacimiento
##   <chr>                 <chr>        <chr>              <dbl>
## 1 Pérez, María         Valparaíso +56 981497134        2001
## 2 Manuel Garrido      valparaíso 914230795        1998
## 3 Soto, Loreto        valparaíso 925479238          2007
## 4 Pérez, Pedro        valparaíso 985740293          2002
## 5 González, Camila    Valparaíso 56 973629584        2004
## 6 Cayuqueo, Marina    Valparaíso 973629853          1999
## 7 Jaime Espinoza      Valparaíso 56981359284        1987
## 8 Hernández, Francisca Valparaíso 56993418209        1971
## 9 González, Olivia    Valparaíso no tengo    1949
    
```

Figura 49. Uso de expresiones regulares, en filtros.

En la figura 50, se realiza una transformación de datos usando la función **mutate**, en donde se requiere que los nombres de ciudad Valparaíso estén correctamente escritos.

```

telefonos %>%
  mutate(ciudad_limpia = case_when(
    str_detect(ciudad, "[v|V]alpara[i|i]so") ~ "Valparaíso"
  ))
    
```

```

## # A tibble: 19 x 5
##   nombre                ciudad      numero_telefonico nacimiento ciudad_limpia
##   <chr>                 <chr>        <chr>              <dbl> <chr>
## 1 Pérez, María         Valparaíso +56 981497134        2001 Valparaíso
## 2 Juan González       Olmué      928374591          1987 <NA>
## 3 Hernández, Lautaro  quilpué    56 964379583        1988 <NA>
## 4 Lobos, Antonia      La Serena 987654321          2001 <NA>
## 5 Martínez, José      La Serena +56-983726153        1996 <NA>
## 6 Manuel Garrido      valparaíso 914230795          1998 Valparaíso
## 7 Soto, Loreto        valparaíso 925479238          2007 Valparaíso
## 8 Pérez, Pedro        valparaíso 985740293          2002 Valparaíso
## 9 González, Camila    Valparaíso 56 973629584        2004 Valparaíso
## 10 Jiménez, Luis      Olmué      925403948          1999 <NA>
## 11 Sanhueza, Pedro    quilpué    56 981408973        1982 <NA>
## 12 Aníbal García      quilpué    982733747          1987 <NA>
## 13 Gómez, Lirayén     quilpué    914257483          2003 <NA>
## 14 Cayuqueo, Marina    Valparaíso 973629853          1999 Valparaíso
## 15 Jaime Espinoza      Valparaíso 56981359284        1987 Valparaíso
## 16 Hernández, Francisca Valparaíso 56993418209        1971 Valparaíso
## 17 López, Carla        Serena     56912338475        1949 <NA>
## 18 Soto, Martina       Serena     925653948          1948 <NA>
## 19 González, Olivia    Valparaíso no tengo    1949 Valparaíso
    
```

Figura 50. Transformar datos con mutate, y expresiones regulares.

En la figura 51, se realiza otro ejemplo de transformación donde se usa nuevamente la función `mutate`, y se realiza la eliminación de cualquier signo +, -, o espacios en blanco.

```
telefonos <- telefonos %>%
  mutate(
    numero_limpio = str_replace_all(numero_telefonico, "\\+|-|[:space:]", "")
  )
telefonos
```

Figura 51. Transformar datos con `mutate`, y expresiones regulares.

En la figura 52, se realiza otra transformación donde: a cualquier combinación menor de 9 dígitos le agregamos el código de país (56) y en el caso notengo, agregamos 000000000000.

```
telefonos <- telefonos %>%
  mutate(
    numero_limpio = str_replace_all(numero_limpio, "^\\d{9}$", paste0("56", numero_limpio)),
    numero_limpio = str_replace_all(numero_limpio, "notengo", "000000000000")
  )
telefonos
```



```
## # A tibble: 19 x 6
```

##	nombre	ciudad	numero_telefoni-	nacimiento	ciudad_limpia	numero_limpio
##	<chr>	<chr>	<chr>	<dbl>	<chr>	<chr>
##	1 Pérez, María	Valpara-	+56 981497134	2001	Valparaíso	56981497134
##	2 Juan Gonzál-	Olmué	928374591	1987	Olmué	56928374591
##	3 Hernández, ~	quilpué	56 964379583	1988	Olmué	56964379583
##	4 Lobos, Anto~	La Sere~	987654321	2001	La Serena	56987654321
##	5 Martínez, J~	La Sere~	+56-983726153	1996	La Serena	56983726153
##	6 Manuel Garr~	valpara~	914230795	1998	Valparaíso	56914230795
##	7 Soto, Loreto	valpara~	925479238	2007	Valparaíso	56925479238
##	8 Pérez, Pedro	valpara~	985740293	2002	Valparaíso	56985740293
##	9 González, C~	Valpara~	56 973629584	2004	Valparaíso	56973629584
##	10 Jiménez, Lu~	Olmué	925403948	1999	Olmué	56925403948
##	11 Sanhueza, P~	quilpué	56 981408973	1982	Olmué	56981408973
##	12 Aníbal Garc~	quilpue	982733747	1987	Olmué	56982733747
##	13 Gómez, Lira~	quilpué	914257483	2003	Olmué	56914257483
##	14 Cayuqueo, M~	Valpara~	973629853	1999	Valparaíso	56973629853
##	15 Jaime Espin~	Valpara~	56981359284	1987	Valparaíso	56981359284
##	16 Hernández, ~	Valpara~	56993418209	1971	Valparaíso	56993418209
##	17 López, Carla	Serena	56912338475	1949	La Serena	56912338475
##	18 Soto, Marti~	Serena	925653948	1948	La Serena	56925653948
##	19 González, O~	Valpara~	no tengo	1949	Valparaíso	000000000000

Figura 52. Transformar datos con `mutate`, y expresiones regulares.

También es posible aplicar transformaciones a variables numéricas. En el ejemplo de la figura 53, se asigna categorías según el año de nacimiento, usando la función `between()`.

```
telefonos <- telefonos %>%  
  mutate(categoria = case_when(  
    between(nacimiento, 2002, 2020) ~ "Jóven",  
    between(nacimiento, 1956, 2001) ~ "Adulto",  
    between(nacimiento, 1955, 1920) ~ "Adulto mayor",  
    negate = TRUE ~ "Centenario"  
  ))  
telefonos
```

Figura 53. Transformar datos con `mutate`, `between`, y expresiones regulares.

El proceso de transformación de atributos, La transformación de datos engloba, en realidad, cualquier proceso que modifique la forma de los datos. Por transformación entendemos aquellas técnicas que transforman un conjunto de atributos en otros, o bien derivan nuevos atributos, o bien cambian el tipo o el rango. La selección de atributos (eliminar los menos relevantes) en realidad no transforma atributos y, en consecuencia, no entra en este grupo de técnicas.

2.3.2 Ordenar los datos usando el paquete `tidyr`

El paquete `tidyr` brinda muchas funcionalidades para realizar transformaciones en un data frame con el fin de poder transformarlo a la estructura que deseemos (Anderson, 2016). El primer paso para esto es averiguar en el dataset cuáles son las variables y cuáles son las observaciones. Una vez que las hemos identificado, nos enfrentamos a tres tipos de problemas:

- Una variable está dividida en múltiples columnas.
- Una observación está dispersa en múltiples filas.
- Múltiples variables están medidas en una única celda.

Usualmente un dataset solo tendrá unos de estos problemas. Para remediar esto deberemos utilizar las funciones `gather`, `spread`, `separate`, y `unite`.

La función **`gather`** toma múltiples columnas y las une en pares clave-valor. Esto permite resolver las situaciones en que tenemos columnas que realmente no

representan variables, sino valores de una variable. Para explicar un ejemplo, se usará el data frame de la figura 54.

```
df
  Provincia Censo_1991 Censo_2001 Censo_2010
Buenos Aires  10934727  11460575  13596320
  Cordoba      1208554   1368301   1466823
  Rosario      1118905   1161188   1236089
```

Figura 54. Data frame de prueba, para transformaciones con gather.

Las columnas Censo_1991, Censo_2001 y Censo_2010 representan a cada censo (o el año en que se realizó cada censo), es decir, el valor de una variable nominal que podría llamarse censo, y cada fila representa tres observaciones, y no una sola. Para ordenar este dataset necesitamos emplear la función gather.

```
gather(data = df, key = "censo", value = "poblacion", 2:4)
  Provincia      censo poblacion
Buenos Aires Censo_1991  10934727
  Cordoba Censo_1991   1208554
  Rosario Censo_1991   1118905
Buenos Aires Censo_2001  11460575
  Cordoba Censo_2001   1368301
  Rosario Censo_2001   1161188
Buenos Aires Censo_2010  13596320
  Cordoba Censo_2010   1466823
  Rosario Censo_2010   1236089
```

Figura 55. Ordenar dataset con gather.

Como resultado, la función toma el data frame df, y une las columnas 2:4 (las que tienen los valores obtenidos en cada censo), metiendo el resultado en el par clave (censo) – valor (población).

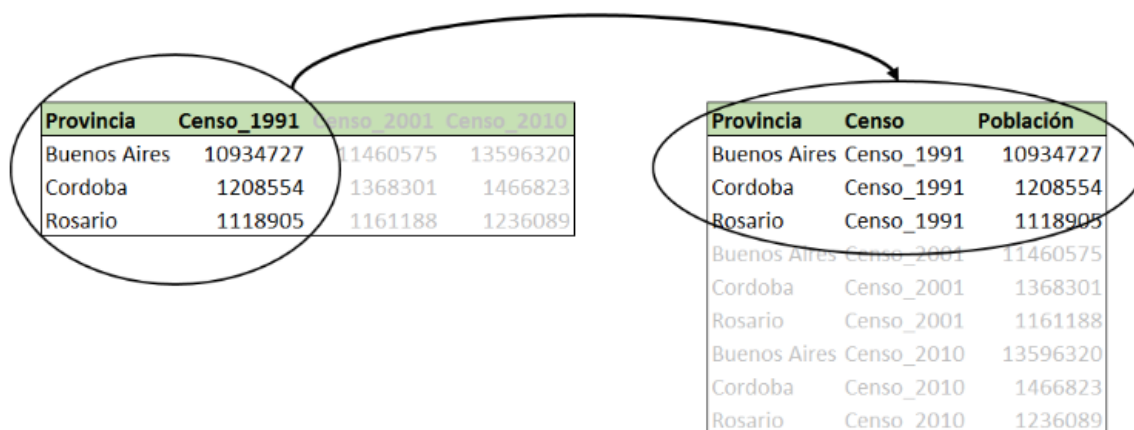


Figura 56. Ordenar dataset con gather.

La función **spread** es usada cuando tenemos una observación dispersa en múltiples filas. En el siguiente ejemplo tenemos un dataset donde una observación es el resultado de un censo, pero cada observación (Censo) está esparcido en dos filas (una fila para la población total, y otro para la población urbana).

```

df
      Pais Censo  Tipo  Población
Argentina 1980 urbana 23.198.068
Argentina 1980 total 27.949.480
Argentina 1991 urbana 28.832.127
Argentina 1991 total 32.615.528
Argentina 2001 urbana 32.380.296
Argentina 2001 total 36.260.130
Argentina 2010 urbana 36.907.728
Argentina 2010 total 40.117.096
    
```

Figura 57. Data frame de prueba, para transformaciones con spread.

Para llevar este dataset a la forma de una observación por fila debemos usar la función **spread**. Tal como se presenta en la figura 58.

```

spread(data = df, key = Tipo, value = Población)
      Pais Censo      total      urbana
Argentina 1980 27.949.480 23.198.068
Argentina 1991 32.615.528 28.832.127
Argentina 2001 36.260.130 32.380.296
Argentina 2010 40.117.096 36.907.728
    
```

Figura 58. Ordenar dataset con spread.

La columna que contiene los nombres de las variables corresponde al argumento key, mientras que la columna que contiene el valor de la variable es value.



Pais	Censo	Tipo	Población
Argentina	1980	urbana	23.198.068
Argentina	1980	total	27.949.480
Argentina	1991	urbana	28.832.127
Argentina	1991	total	32.615.528
Argentina	2001	urbana	32.380.296
Argentina	2001	total	36.260.130
Argentina	2010	urbana	36.907.728
Argentina	2010	total	40.117.096

Pais	Censo	total	urbana
Argentina	1980	27.949.480	23.198.068
Argentina	1991	32.615.528	28.832.127
Argentina	2001	36.260.130	32.380.296
Argentina	2010	40.117.096	36.907.728

Figura 59. Ordenar dataset con spread.

Como se observa, la función spread hace lo opuesto a gather. Estas son funciones complementarias, es decir, si al resultado de aplicar la función spread le aplicamos la función gather llegamos al dataset original. Otra observación interesante es que gather alarga los data frames, mientras que spread los hace más ancho.

La función **separate** divide una columna en múltiples columnas, tomando como separador algún símbolo. En la figura 60, tenemos una columna (Tasa.de.población.urbana) que contiene más de un valor. Para solucionar esto tenemos que emplear la función separate.

```

df
  Pais Censo Tasa.de.población.urbana
Argentina 1980 23198068 / 27949480
Argentina 1991 28832126 / 32615528
Argentina 2001 32380296 / 36260130
Argentina 2010 36907728 / 40117096

separate(data = df,
         col = Tasa.de.población.urbana,
         into = c("urbana", "total"),
         sep = "/")

  Pais Censo urbana total
Argentina 1980 23198068 27949480
Argentina 1991 28832126 32615528
Argentina 2001 32380296 36260130
Argentina 2010 36907728 40117096
    
```

Figura 60. Ordenar dataset con separate.

Por defecto, la función **separate** usa como separador cualquier valor no alfa-numérico. En este ejemplo he indicado explícitamente el separador (sep) solo para claridad. Por otro lado, el tipo de dato que se asigna a las nuevas columnas es el mismo que el original. Como es de esperar, las columnas resultantes de la división son de tipo character, pero dado su significado lo que deseamos es que sean tratadas como numeric o integer. Para indicar esto podemos hacer la conversión del tipo de dato manualmente, o se puede agregar la opción `convert = TRUE` a la función `separate`.

```
str(separate(data = df,
              col = Tasa.de.población.urbana,
              into = c("urbana", " total"),
              sep = "/",
              convert = TRUE))
```

'data.frame': 4 obs. of 4 variables:

```
$ Pais : chr "Argentina" "Argentina" "Argentina" "Argentina"
$ Censo : chr "1980" "1991" "2001" "2010"
$ urbana: num 23198068 28832126 32380296 36907728
$ total: int 27949480 32615528 36260130 40117096
```

Figura 61. Ordenar dataset con `separate`.

La función **unite** complementa a `separate`. Esta función lo que hace es tomar múltiples columnas (...) y las colapsa en una única columna (col), separando los elementos mediante sep. En la figura 62, se presenta un ejemplo, donde se unen los atributos día, mes y año, en una variable fecha.

<pre>df</pre> <table border="1"> <thead> <tr> <th>ID</th> <th>dia</th> <th>mes</th> <th>anio</th> </tr> </thead> <tbody> <tr><td>1</td><td>25</td><td>9</td><td>2015</td></tr> <tr><td>2</td><td>30</td><td>7</td><td>2015</td></tr> <tr><td>3</td><td>7</td><td>3</td><td>2015</td></tr> <tr><td>4</td><td>15</td><td>8</td><td>2015</td></tr> </tbody> </table>	ID	dia	mes	anio	1	25	9	2015	2	30	7	2015	3	7	3	2015	4	15	8	2015		<pre>unite(data = df, col = Fecha, sep = "-", dia,mes,anio)</pre> <table border="1"> <thead> <tr> <th>ID</th> <th>Fecha</th> </tr> </thead> <tbody> <tr><td>1</td><td>25-9-2015</td></tr> <tr><td>2</td><td>30-7-2015</td></tr> <tr><td>3</td><td>7-3-2015</td></tr> <tr><td>4</td><td>15-8-2015</td></tr> </tbody> </table>	ID	Fecha	1	25-9-2015	2	30-7-2015	3	7-3-2015	4	15-8-2015
ID	dia	mes	anio																													
1	25	9	2015																													
2	30	7	2015																													
3	7	3	2015																													
4	15	8	2015																													
ID	Fecha																															
1	25-9-2015																															
2	30-7-2015																															
3	7-3-2015																															
4	15-8-2015																															

Figura 62. Ordenar dataset con `unite`.

2.3.3 Reducción de dimensionalidad

Si tenemos muchas dimensiones (atributos) respecto a la cantidad de instancias, pueden existir demasiados grados de libertad, por lo que los patrones extraídos pueden ser poco robustos. Una manera de intentar resolver este problema es mediante la reducción de dimensiones. La reducción se puede realizar por selección de un subconjunto de atributos, o bien la sustitución del conjunto de atributos iniciales por otros diferentes (Hernández & Ferri, 2022).

Análisis de componentes principales (PCA):

La técnica más conocida para reducir la dimensionalidad por transformación se denomina “análisis de componentes principales” (“principal component analysis”), PCA. PCA transforma los m atributos originales en otro conjunto de atributos p donde $p \leq m$. Este proceso se puede ver geoméricamente como un cambio de ejes en la representación (proyección). Los nuevos atributos se generan de tal manera que son independientes entre sí y, además, los primeros tienen más relevancia (más contenido informacional) que los últimos.

Ejemplo: Dataset Segment, 19 Atributos El método Principal Components genera 10 atributos cubriendo el 97% de la varianza. También es posible conocer la evolución de la precisión del algoritmo J48, árbol de decisión, dependiendo del número de atributos. En la figura 63, se presenta este resultado de reducción de dimensionalidad.

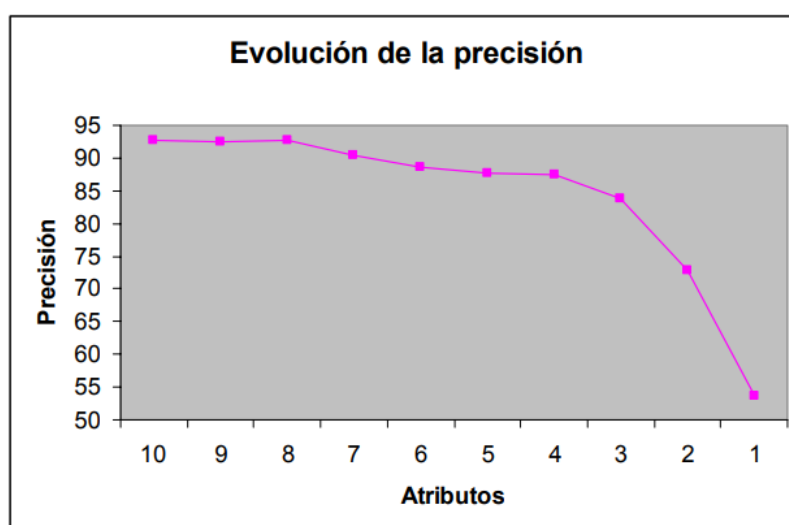


Figura 63. Análisis de componentes principales, reducción de dimensionalidad.

2.3.4 Aumento de dimensionalidad

En otras ocasiones añadir atributos nuevos puede mejorar el proceso de aprendizaje.

- La regresión lineal no se aproxima a la solución
- Añadiendo un nuevo atributo $z = \text{meses}^2$ se obtiene un buen modelo

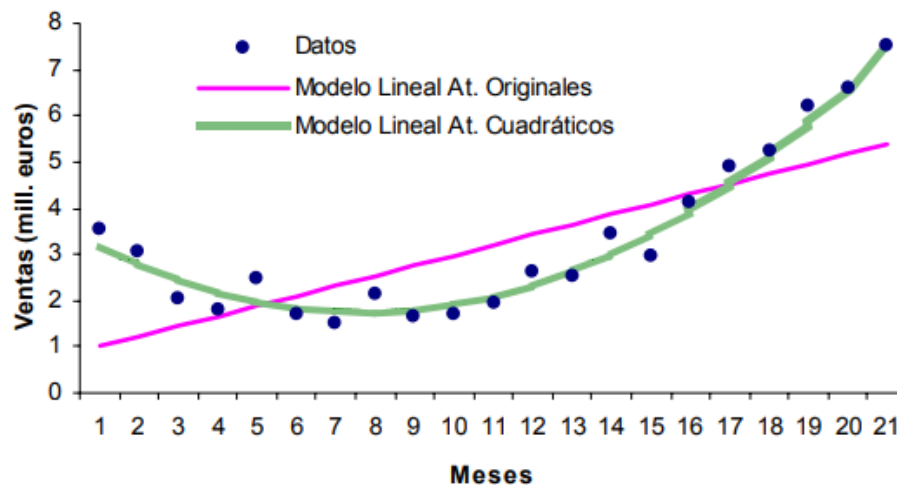


Figura 64. Análisis de componentes principales, aumento de dimensionalidad.

En el aumento de la dimensionalidad, se aplican técnicas de generación de nuevos atributos:

- **Numéricos:** Se utilizan, generalmente, operaciones matemáticas básicas de uno o más argumentos
- **Nominales:** Operaciones lógicas: conjunción, disyunción, negación, implicación, condiciones M-de-N (M-de-N es cierto si y sólo si al menos M de las N condiciones son ciertas), igualdad o desigualdad.

El conocimiento del dominio es el factor que más determina la creación de buenos atributos derivados, en la siguiente tabla se presenta algunos escenarios.

Tabla 1. Aumento de dimensionalidad, atributos derivados.

Atributo Derivado	Fórmula
Índice de obesidad	$\text{Altura}^2 / \text{peso}$
Hombre familiar	Casado, varón e "hijos>0"
Síntomas SARS	3-de-5 (fiebre alta, vómitos, tos, diarrea, dolor de cabeza)
Riesgo póliza	X-de-N (edad < 25, varón, años de carné < 2, vehículo deportivo)
Beneficios brutos	Ingresos - Gastos
Beneficios netos	Ingresos – Gastos – Impuestos
Desplazamiento	Pasajeros * kilómetros
Duración media	Segundos de llamada / número de llamadas
Densidad	Población / Área
Retardo compra	Fecha compra – Fecha campaña

2.3.4 Discretización de valores

La discretización, o cuantización (también llamada “binning”) es la conversión de un valor numérico en un valor nominal ordenado. La discretización se debe realizar cuando:

- El error en la medida puede ser grande.
- Existen umbrales significativos (p.e. notas).
- En ciertas zonas el rango de valores es más importante que en otras (interpretación no lineal).
- Aplicar ciertas tareas de MD que sólo soportan atributos nominales (p.e. reglas de asociación).

Las tablas de contingencias se usan normalmente sobre variables discretas, no numéricas, ya que estas últimas tienden a producir tablas muy grandes que dificultan el análisis. Por ejemplo, al tener muchos valores distintos en la variable `iris$Sepal.Length`, es donde podemos discretizar la variable continua y obtener una serie de rangos que, pueden ser tratados como variables discretas. La función a usar en este caso es **cut()**, obteniendo una transformación de la variable original que después sería usada con la función **table()**.

La sintaxis, **cut(vector, breaks = cortes)** discretiza los valores contenidos en el vector entregado como primer parámetro según lo indicado por el argumento

breaks. Este puede ser un número entero, en cuyo caso se harán tantas divisiones como indique, o bien un vector de valores que actuarían como puntos de corte.

En el ejercicio de la figura 65, se muestra cómo discretizar la variable **iris\$Sepal.Length** a fin de obtener una tabla de contingencia más compacta, de la cual es fácil inferir como cambia la longitud de sépalo según la especie de la flor.

```

> # Discretizar la longitud de sépalo
> cortes <- seq(from=4, to=8, by=0.5)
> seplen <- cut(iris$Sepal.Length, breaks = cortes)
> # Usamos la variable discretizada con table()
> table(seplen, iris$Species)
    
```

seplen	setosa	versicolor	virginica
(4,4.5]	5	0	0
(4.5,5]	23	3	1
(5,5.5]	19	8	0
(5.5,6]	3	19	8
(6,6.5]	0	12	19
(6.5,7]	0	8	10
(7,7.5]	0	0	6
(7.5,8]	0	0	6

Figura 65. Tabla de contingencia sobre valores discretizados.



Videos: En el siguiente enlace podrán encontrar y observar un video sobre el proceso de instalación de R y R Studio: <https://rpubs.com/Edimer/675756>



Lectura complementaria de la Unidad 1: Tipos de datos en R
<https://rpubs.com/Edimer/754932>

Bibliografía:

Aggarwal, C. C. (2015). *Data Mining*. <https://doi.org/10.1007/978-3-319-14142-8>

Anderson, M. (2016). *Manipulación de data frames con tidy*.

<https://mauricioanderson.com/curso-r-tidyr/>

Ayala, J. (2020, marzo 2). *Extracción y selección de atributos*.

<https://rpubs.com/JairoAyala/580877>

Bello, E. (2021). ¿Qué es el minado de Datos o Data Mininig? Técnicas y pasos a seguir. *Thinking for Innovation*, 1–8. <https://www.iebschool.com/blog/data-mining-mineria-datos-big-data/>

Calderón Méndez, N. de J. (2006). *Minería de datos, una herramienta para la toma de decisiones*.

Charte, F. (2014). *Análisis exploratorio y visualización de datos con R*. 23–26.

<http://www.fcharte.com/libros/ExploraVisualizaConR-Fcharte.pdf>

Clinic Cloud. (2016). *¿Qué es el data mining? La definición de la minería de datos*.

<https://clinic-cloud.com/blog/data-mining-que-es-definicion-mineria-de-datos/>

Hernández Orallo, J., & Ferri Ramirez, C. (2022). *Minería de Datos y Extracción de Conocimiento de Bases de Datos*.

<http://josephorallo.webs.upv.es/docent/doctorat/t2a.pdf>

Mendoza Vega, J. B. (2018). *R para principiantes*.

<https://bookdown.org/jboscomendoza/r-principiantes4/na-y-null.html>

Microsoft. (2022, abril 20). *Conceptos de minería de datos*.

<https://docs.microsoft.com/es-es/analysis-services/data-mining/data-mining-concepts?view=asallproducts-allversions>

Quintela del Rio, A. (2019). *Variables discretas y continuas | Estadística Básica*

Edulcorada. <https://bookdown.org/aquintela/EBE/variables-discretas-y-continuas.html>

Quiroz Gil, N. L. (2012). *Aplicación del proceso de KDD en el contexto de*

bibliomining: El caso Elogim. (Spanish). *Revista Interamericana de Bibliotecología*, 35(1), 97–108.

<http://aprendeenlinea.udea.edu.co/revistas/index.php/RIB/article/view/13341/11937>

Salazar, C. (2022). *Limpieza de datos*. <https://rpubs.com/camilamila/limpieza>

Timaran, S. R., Hernandez, I., Caicedo, S. J., Hidalgo, A., & Alvarado, J. C. (2016). El proceso de descubrimiento de conocimiento en bases de datos. *Ingenierías*, 8(26), 37–47.

Vallejo Ballesteros, H. F., Guevara Iñiguez, E., & Medina Velasco, S. R. (2018). Minería de Datos. *RECIMUNDO: Revista Científica de la Investigación y el Conocimiento*, 2, 339–349. <https://doi.org/10.26820/recimundo/2.esp.2018.339-349>

Wickham, H., & Garrett, G. (2017). *R for Data Science*.